A Heuristic and Deadlock-Aware Search System for Sokoban Puzzle Solving

A PROJECT REPORT

Submitted by

METTU MANI CHANDHAN SAI (241901054)

MEBISH V D (241901053)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)



RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institute

CHENNAI

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report "A HEURISTIC AND DEADLOCK-AWARE SEARCH SYSTEM FOR SOKOBAN PUZZLE SOLVING" is the bonafide work of "METTU MANI CHANDHAN SAI (241901054) and MEBISH V D (241901053)" who carried out the project work under my supervision.

SIGNATURE

Dr. Bhuvana S, Ph.D.,

ASSISTANT PROFESSOR(SG)
Department of Computer Science
and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

Sokoban is a warehouse puzzle in which a keeper pushes crates onto goal squares. Its rules are simple, yet the puzzle is PSPACE-complete, and a single bad push can leave a level that can never be finished. This project presents Sokoban Search Lab, a browser-based system that solves Sokoban with classical artificial intelligence search and measures how much each search technique contributes. The puzzle is modelled as a deterministic state space in which a state is the set of crate positions together with the region the keeper can walk to, and an action is a single push. Four solvers are built on this model, each adding one idea to the previous one: Breadth-First Search as the baseline, A* with a Manhattan-distance heuristic, A* with deadlock rules (corner, wall-line, dead square and freeze), and A* with a matching heuristic computed by the Hungarian algorithm. On a set of 34 levels, all four solvers return the same minimum number of pushes, but the amount of search differs greatly. Breadth-First Search expanded 1,774,999 positions in total and finished only 27 levels within its budget, while the final solver finished all 34 levels after expanding just 4,930 positions, a median saving of 99.5 %. The same Python code runs inside the browser through Pyodide, so the game, hints, live races between algorithms, the benchmark and the level editor share one implementation, with validated input and a limit on the running time of every search.

ACKNOWLEDGEMENT

First and foremost, we thank Almighty God for enabling us to complete this project successfully. We express our sincere gratitude to our Chairman, Mr. S. Meganathan, B.E., F.I.E., for his dedicated efforts in providing quality education at this esteemed institution. We are deeply grateful to our beloved Chairperson, Dr. Thangam Meganathan, Ph.D., for her constant motivation and encouragement, which inspired us throughout the completion of this project. We also thank our Vice Chairman, Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the necessary infrastructure and facilities.

We express our sincere gratitude to our College Principal, Dr. S. N. Murugesan, M.E., Ph.D., and Dr. P. Tamilselvi, Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security), for their support and encouragement.

We are especially thankful to our project guide, Dr. Bhuvana S., Ph.D., Assistant Professor(SG), for her valuable guidance, continuous encouragement, and support throughout the project. Finally, we thank our parents and friends for their direct and indirect contributions, encouragement, and support in the successful completion of this project.

METTU MANI CHANDHAN SAI (241901054)
MEBISH V D (241901053)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	4
	1.3 OBJECTIVES	5
	1.4 SCOPE	5
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	6
	1.6 SDG ALIGNMENT	7
2	**LITERATURE REVIEW**	**9**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	9
	2.1.1 LITERATURE SURVEY	9
	2.1.2 DISCUSSION OF THE LITERATURE	11
	2.1.3 EXISTING SYSTEM AND ITS SECURITY APPROACHES	11
	2.2 RESEARCH GAPS	12
	2.3 PROPOSED SYSTEM	13
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	13
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	15
	2.3.3 ADVANTAGES OF THE PROPOSED SYSTEM	16
3	**SYSTEM DESIGN**	**17**
	3.1 SYSTEM ARCHITECTURE	17

	3.2	MODULE DESIGN	20
	3.3	DATA FLOW / UML DIAGRAMS	23
	3.4	THREAT MODEL AND ATTACK SURFACE	29
	3.5	SECURITY CONTROLS	32
4	**SYSTEM DESCRIPTION**		**34**
	4.1	DATA COLLECTION	34
	4.2	DATA PREPROCESSING	37
		4.2.1 ALGORITHMS / METHODOLOGY	40
		4.2.2 RESULTS AND DISCUSSION	45
5	**CONCLUSION**		**57**
	FUTURE WORK (PHASE II)		58
	REFERENCES		**59**
	APPENDIX		**61**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	PEAS DESCRIPTION OF THE SOKOBAN AGENT	3
1.2	SECURITY REQUIREMENTS OF THE SYSTEM	6
1.3	SDG ALIGNMENT OF THE PROJECT	7
2.1	EXISTING SYSTEMS COMPARED WITH THE PROPOSED SYSTEM	12
3.1	HARDWARE REQUIREMENTS	19
3.2	SOFTWARE REQUIREMENTS	19
3.3	MODULE DESCRIPTION	22
3.4	ASSETS AND SECURITY OBJECTIVES	31
3.5	STRIDE THREAT ANALYSIS	31
3.6	IMPLEMENTED SECURITY CONTROLS	32
4.1	LEVEL CORPUS BY PACK	34
4.2	METRICS RECORDED FOR EVERY RUN	36
4.3	DEAD SQUARES PRECOMPUTED PER PACK	39
4.4	PROPERTIES OF THE FOUR ALGORITHMS	44
4.5	AUTOMATED TEST SUITE RESULTS	45
4.6	BENCHMARK SUMMARY OVER ALL 34 LEVELS	46
4.7	POSITIONS EXPANDED ON REPRESENTATIVE LEVELS	47
4.8	POSITIONS EXPANDED AND TIME PER PACK	49
4.9	SOLUTION LENGTH IN KEEPER MOVES, SHOWDOWN PACK	50
A.1	POSITIONS EXPANDED ON ALL 34 LEVELS	62

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	SOKOBAN GAME ELEMENTS AND RULES OF MOVEMENT	1
1.2	GOAL-BASED SEARCH AGENT FOR SOKOBAN AND ITS ENVIRONMENT PROPERTIES	3
2.1	BLOCK DIAGRAM OF THE PROPOSED SYSTEM	14
2.2	THE FOUR-RUNG ALGORITHM LADDER WITH MEASURED TOTALS	15
3.1	LAYERED SYSTEM ARCHITECTURE	18
3.2	MODULE DESIGN AND DEPENDENCY ORDER	21
3.3	DATA FLOW DIAGRAM – LEVEL 0 (CONTEXT DIAGRAM)	23
3.4	DATA FLOW DIAGRAM – LEVEL 1	24
3.5	USE CASE DIAGRAM	25
3.6	ACTIVITY DIAGRAM OF A* SEARCH WITH DEADLOCK PRUNING	26
3.7	SEQUENCE DIAGRAM OF A SOLVE REQUEST	27
3.8	CLASS DIAGRAM OF THE CORE TYPES	28
3.9	THREAT MODEL: TRUST BOUNDARIES AND ATTACK SURFACE	30
4.1	SAMPLE LEVELS FROM THE FOUR PACKS	35
4.2	LEVEL GENERATION BY REVERSE PLAY AND SOLVER GRADING	36
4.3	DATA PREPROCESSING PIPELINE FROM XSB TEXT TO (LEVEL, STATE)	37
4.4	PRECOMPUTED DEAD-SQUARE MAPS (C = CORNER, W = WALL-LINE, N = NO-PATH)	38
4.5	FROM STEP-LEVEL TO PUSH-LEVEL STATES BY KEEPER-REGION NORMALISATION	40
4.6	THE FOUR DEADLOCK RULES (RED OUTLINE = CRATE IN DEADLOCK)	42
4.7	MANHATTAN VERSUS HUNGARIAN MATCHING HEURISTIC ON THE SAME POSITION	43

4.8	TOTAL POSITIONS EXPANDED AND TOTAL SEARCH TIME (LOG SCALES)	46
4.9	POSITIONS EXPANDED PER LEVEL, WAREHOUSE AND SHOWDOWN PACKS	47
4.10	EFFORT RELATIVE TO BFS ON EACH LEVEL (LOG SCALE)	48
4.11	PUSHES PRUNED PER RULE AND DEAD SQUARES PER RULE OVER ALL LEVELS	49
4.12	SEARCH EFFORT VERSUS NUMBER OF CRATES (MEDIANS AND INDIVIDUAL LEVELS)	50
4.13	PLAY VIEW OF THE WEB APPLICATION (STOREROOM, CLASSIC PACK)	51
4.14	DEAD-SQUARE OVERLAY ON PALLET MAZE	52
4.15	DEADLOCK COACH NAMING A CORNER TRAP	52
4.16	AI HINT MARKING THE NEXT OPTIMAL PUSH	53
4.17	BFS SEARCH IN PROGRESS WITH THE SEARCH-TRACE OVERLAY	54
4.18	SOLVED LEVEL WITH TIMELINE AND SEARCH STATISTICS	54
4.19	RACE VIEW: FOUR ALGORITHMS ON TIGHT TURN	55
4.20	LAB VIEW: BENCHMARK OF THE CLASSIC PACK IN THE BROWSER	56
A.1	LEVEL EDITOR AND GENERATOR (BUILD VIEW)	64
A.2	BUILT-IN GUIDE EXPLAINING THE GAME, ALGORITHMS AND METRICS	65

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Sokoban, which means “warehouse keeper” in Japanese, was designed by Hiroyuki Imabayashi and released by Thinking Rabbit in 1982. The board is a grid of walls and floor squares. Some floor squares are goals, there are as many crates as goals, and a single keeper moves up, down, left or right, one square at a time. When the keeper walks into a crate, the crate moves one square ahead, but only if that square is empty floor. Crates cannot be pulled, and two crates in a row cannot be pushed together. A level is finished when every crate rests on a goal. Figure 1.1 shows the elements of the game on a small board, together with the four rules of movement as they are implemented in this project.

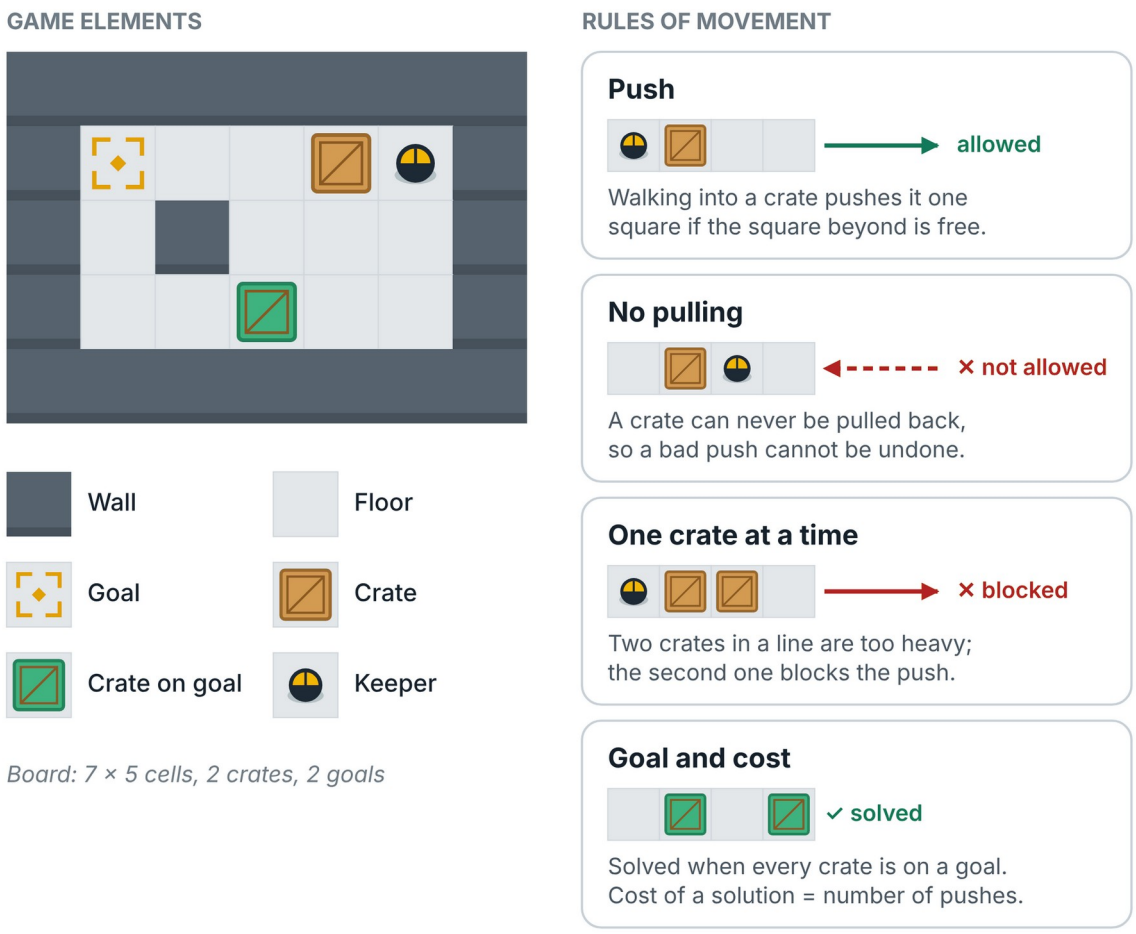


Figure 1.1 Sokoban Game Elements and Rules of Movement

The rules are short, but solving the puzzle by computer is hard. Culberson proved that deciding whether a Sokoban level has a solution is PSPACE-complete [1], and Dor and Zwick showed that even restricted versions are NP-hard [2]. In practice the difficulty has two sources. The number of reachable positions grows very quickly as crates are added and rooms become larger. In addition, pushes cannot be undone. A crate pushed into a corner, or against a wall with no goal along it, is stuck for good. Such a position is called a *deadlock*, and a search algorithm that cannot recognise deadlocks spends most of its time on positions that are already lost [3], [4]. For these reasons Sokoban has served for over twenty years as a test problem for single-agent search and planning, and more recently for deep reinforcement learning [5].

Sokoban was chosen for this Foundations of Artificial Intelligence project because it matches the problem-solving agent model of Russell and Norvig [6] very closely. The environment is fully observable, deterministic, static and discrete, there is a single agent, and the rules are completely known. The agent therefore has nothing to learn and no uncertainty to handle, and its performance depends only on how it searches. The search is improved in three steps covered in the course: a heuristic estimate of the remaining cost (A* [7]), domain knowledge that discards hopeless states, and finally a stronger heuristic that is still admissible. Figure 1.2 shows the goal-based agent built in this project: the environment at the top sends percepts to the state model, the search strategy combines the goal test with heuristic and deadlock knowledge, and the resulting plan is sent back as actions. The properties of the environment are listed below the agent.

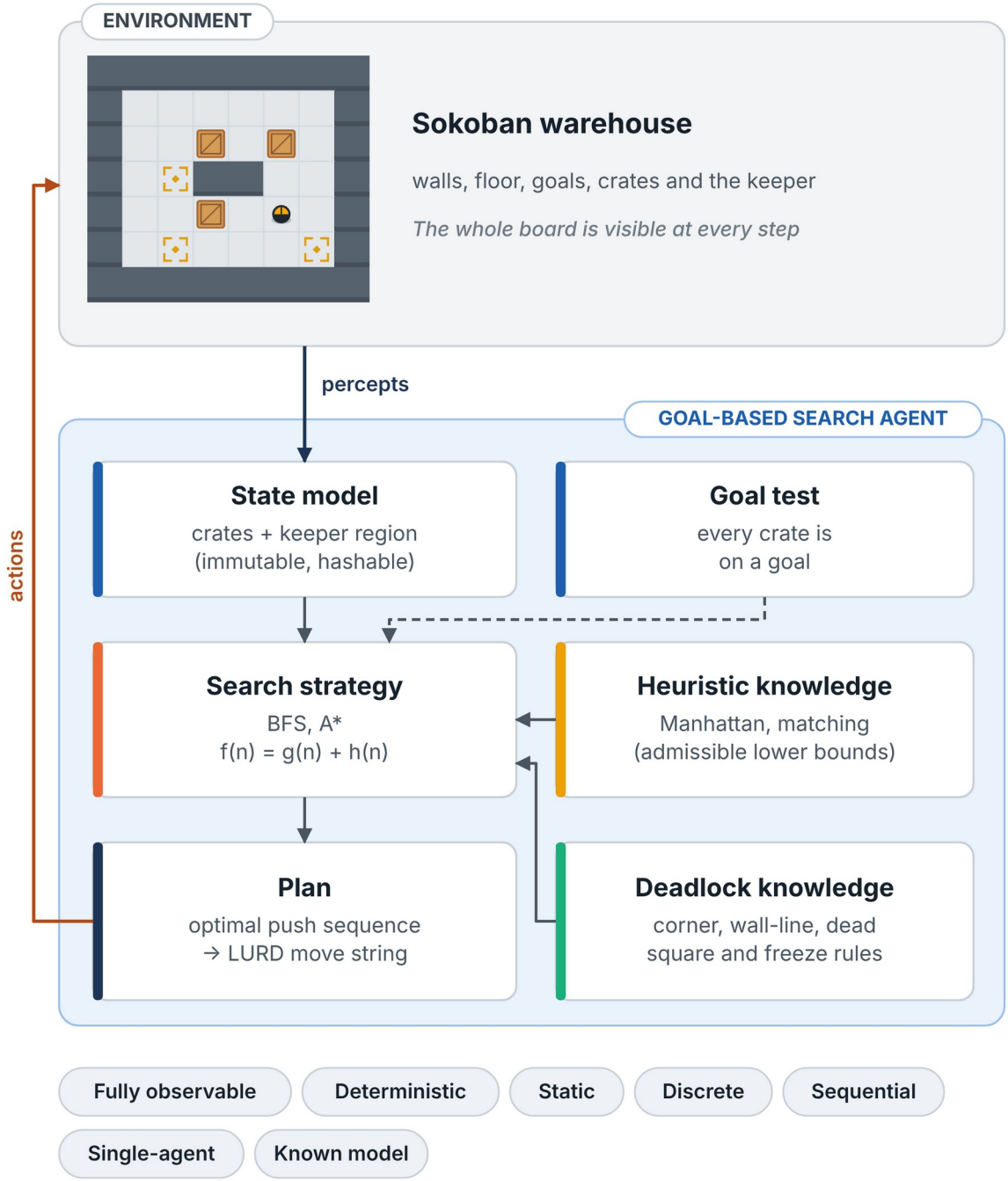


Figure 1.2 Goal-Based Search Agent for Sokoban and its Environment Properties

The agent can also be described with the PEAS framework. Table 1.1 gives its performance measure, environment, actuators and sensors.

Table 1.1 PEAS Description of the Sokoban Agent

Component	Description in this project
Performance measure	Solve the level with the fewest pushes; after that, expand as few positions and use as little time as possible

Component	Description in this project
Environment	A rectangular warehouse grid closed in by walls, with floor, goals, crates and one keeper (XSB level format)
Actuators	Pushing one crate by one square (U, D, L, R); the walk to the pushing position is found by a flood fill and costs nothing
Sensors	The whole board at every step: crate squares, keeper square, walls and goals (fully observable)

Sokoban is also more than a game. A warehouse robot moving pallets, a crane shifting containers in a port yard and parts moving along a factory floor all face the same kind of problem: objects that can only be pushed in certain directions, narrow aisles, and mistakes that are expensive or impossible to reverse. The search and pruning ideas in this report are the same ones used in automated planning for such tasks.

1.2 PROBLEM STATEMENT

To solve a level, a solver must find a sequence of pushes that places every crate on a goal without creating a deadlock along the way. A plain search that explores blindly runs into three problems.

- The number of positions explodes. Even Breadth-First Search over push-level states needs 9,952 positions for the three-crate Storeroom level, and on several levels with five or six crates it used up the whole budget of 200,000 positions without finding the answer.
- A large part of every board is a dead end. Across the 34 levels of this project, 529 of the 1,417 floor squares (37.3 %) are squares from which a crate can never reach a goal. A solver without this knowledge keeps exploring positions that contain such crates.
- The search cannot be seen or compared. Sokoban games allow play but do not show how a computer would solve the level, and most simple solvers only print an answer, with no figures showing how much each technique saved.

The problem addressed in this project is therefore to build a Sokoban solver that stores the puzzle compactly, always finds a solution with the fewest pushes, reduces the search with admissible heuristics and correct deadlock rules, and measures how much each of

these ideas helps under the same conditions. A further aim is to let a person watch the search in a secure web interface that needs no installation.

1.3 OBJECTIVES

1. To represent a Sokoban position as an immutable, hashable state made of the crate positions and the keeper's reachable region, with pushes as the actions, so that repeated states are detected and skipped.
2. To implement Breadth-First Search as the baseline, which always finds a minimum-push solution and provides a fixed reference for every comparison.
3. To implement A* with the Manhattan-distance heuristic and to show that this heuristic is admissible and consistent, so that the solution stays optimal while far fewer positions are expanded.
4. To add deadlock detection (corner, wall-line, dead-square and freeze rules) that removes hopeless pushes before they reach the frontier, and to confirm with exhaustive search that the rules never remove a real solution.
5. To add a stronger heuristic that uses true push distances and a minimum-cost assignment of crates to goals, found with the Hungarian algorithm.
6. To run all four algorithms on the same levels and record positions expanded and generated, pushes pruned by each rule, solution pushes and moves, time, and the saving compared with Breadth-First Search.
7. To build a web application in which a user can play, take hints, watch solutions, race the algorithms, run the benchmark and design new levels.
8. To keep the system secure through validated input, limits on computation, a restricted interface between the page and the solver, safe display of user text and a static deployment with no server.

1.4 SCOPE

Phase I covers the design, implementation and evaluation of the solver and its interface. The system follows the standard Sokoban rules on rectangular, fully walled levels written in the common XSB text format. It contains four optimal solvers, namely Breadth-First Search, A*, A* with deadlock pruning and A* with the matching heuristic, each with a limit on the number of positions and on time. The level set has 34 levels in four packs

(Training, Classic nine, Warehouse and Showdown), and new levels can be drawn in a level editor or produced by a generator that plays the game backwards. The web interface offers Play, Race, Lab, Build and Guide views and runs the same Python solver through WebAssembly. A command-line benchmark writes the results as tables, CSV and Markdown, and an automated test suite checks every module. The scope also includes a security analysis suited to a client-side educational application, with requirements, a threat model and controls.

Some topics are outside the scope of Phase I and are listed as future work in Chapter 5. Large competition sets such as the 90 XSokoban levels are not attempted, since they need pattern databases, macro moves and goal-room analysis [4], [8]. Non-optimal (greedy or weighted) search, learned heuristics, multiplayer or server features and user accounts are also left out.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

Sokoban Search Lab is a puzzle solver, but it is also a web application. It accepts level text typed or pasted by the user, it runs heavy computations in the user's browser, and it is downloaded over the internet. Because this is a project of the Department of Computer Science and Engineering (Cyber Security), security requirements were written down at the start, based on the OWASP Top 10 [9] and the STRIDE threat categories [10], and the design was checked against them. Table 1.2 lists each requirement and how the system meets it. The detailed threat model is given in Section 3.4.

Table 1.2 Security Requirements of the System

ID	Requirement	How the system meets it
SR-1	Validate all untrusted level input before use	`parse_level` accepts only the nine XSB symbols, requires one keeper, at least one crate, as many crates as goals, full enclosure and reachability, and otherwise raises a readable `LevelError`
SR-2	Bound computation (availability)	Every search has a node limit and a time limit (200,000 positions and 20 s in the browser); the generator clamps room size and crate count; benchmark budgets are capped at 30 s
SR-3	Least privilege at the page–solver interface	`bridge.call` runs only public, callable `Game` methods; names starting with an underscore are refused; failures come back as `{error}` instead of an exception

ID	Requirement	How the system meets it
SR-4	Safe output encoding (no script injection)	User-controlled strings (level titles and notes) are written with `textContent`; HTML templates escape values with `esc()`
SR-5	Confidentiality and privacy	No accounts, no personal data, no analytics and no server calls after loading; `localStorage` keeps only theme, sound and last level
SR-6	Integrity of delivered code	Static site served over HTTPS by GitHub Pages; the Python runtime is copied from the official package; a test fails if the published site differs from the sources
SR-7	Isolation of execution	Python runs inside Pyodide (WebAssembly) in the browser sandbox; user text is never passed to `eval`; the solver has no file-system or network access
SR-8	Correctness and integrity of results	59 automated tests, including fuzz tests showing that all algorithms agree on the optimal push count and that every pruned position really is unsolvable

1.6 SDG ALIGNMENT

The project was mapped to the United Nations Sustainable Development Goals [11]. Its main contribution is to education, with smaller links to innovation, productive work and the efficient use of computing resources. Table 1.3 gives this mapping in the format set by the department.

Table 1.3 SDG Alignment of the Project

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 4: Quality Education	4.4: youth and adults with relevant technical and ICT skills	A free lab, with nothing to install, where students can see and measure search, heuristics and pruning
2	SDG 9: Industry, Innovation and Infrastructure	9.5: enhance research and technological capabilities	An open, repeatable benchmark of four search strategies; the methods carry over to robotics and logistics planning
3	SDG 8: Decent Work and Economic Growth	8.2: productivity through technology and innovation	Planning pushes while avoiding deadlocks is the same problem as moving pallets and containers in warehouses

4	SDG 12: Responsible Consumption and Production	12.2: efficient use of natural resources (energy)	Better algorithms instead of more hardware: 0.28 % of the positions of Breadth-First Search and 0.6 s of CPU for all 34 levels, with no GPU or server

CHAPTER 2

LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

This chapter reviews the published work on Sokoban and single-agent search that informed the design of the system, describes how Sokoban is played and solved by existing software, and examines the security approaches that this software relies on.

2.1.1 Literature Survey

Hart, Nilsson and Raphael introduced the A* algorithm in 1968 [7]. A* expands nodes in order of f = g + h, where g is the cost so far and h is an estimate of the remaining cost. They proved that A* returns an optimal path when h never overestimates the true cost, and that no node needs to be expanded twice when h is also consistent. The limitation is that A* is only as good as its heuristic, and its memory use grows with the size of the frontier. This result is the basis of every informed solver in this project.

The hardness of the puzzle was settled by two complexity papers. Culberson showed in 1997 that Sokoban is PSPACE-complete by a reduction from linear-bounded automata [1], and Dor and Zwick showed in 1999 that Sokoban and related motion planning puzzles remain NP-hard even under strong restrictions [2]. These papers do not offer a solver, but they explain why any practical solver must rely on heuristics and pruning rather than on brute force.

Junghanns and Schaeffer studied how standard single-agent search behaves on Sokoban. In their 1998 paper, IDA* with a simple lower-bound heuristic failed on most benchmark levels, and the authors traced the failure to deadlocks [3]. Their later program, Rolling Stone, combined IDA* with deadlock tables, macro moves, relevance cuts and pattern search, and solved a large part of the 90-level XSokoban set [4]. The work showed clearly that domain knowledge is essential, but the program is a large native application, and some of its enhancements give up optimality. Botea, Müller and Schaeffer then divided levels into rooms and tunnels and planned at this higher level, which made the search space much smaller [12]. The abstraction, however, depends on the shape of each level and is hard to explain to learners.

A separate line of work aims at provably optimal solutions. Pattern databases, introduced by Culberson and Schaeffer, store the exact cost of small sub-problems and use them as admissible heuristics [13]. Pereira, Ritt and Buriol applied pattern databases with Sokoban-specific knowledge in 2015 and obtained optimal solutions for benchmark levels that had been out of reach [8]. In 2016, Pereira et al. improved the heuristic with intermediate goal states and a new tie-breaking rule, which raised the number of optimally solved instances from 20 to 28 [14]. These methods are powerful, but they need heavy pre-computation and are too complex for a teaching setting.

Learning-based approaches appeared later. Racanière et al. trained imagination-augmented deep reinforcement learning agents on procedurally generated Sokoban levels in 2017 [5], and Orseau et al. combined a learned policy with tree search (LevinTS) and proved bounds on the number of expansions in 2018 [15]. Both require large training budgets, and neither guarantees an optimal solution.

Venkatesan, Jain and Grewal compared DFS, BFS, uniform-cost search, A* and a convolutional neural network on Sokoban in 2018 [16]. A* with a heuristic that combines Hungarian matching and Manhattan distance was the best of the search methods, while the network reached only about 45 % accuracy on test data. Dynamic deadlock detection was left as future work. In 2020, Shoham and Schaeffer presented FESS, a search guided by an abstract feature space, which became the first program to solve all 90 XSokoban levels [17]. Its solutions are not optimal, and the features it uses are sophisticated. Shoham and Elidan later showed in 2021 that reinforcement learning from reversed play helps on hard instances, at a high computational cost [18].

The two most recent works are the closest to this project. Peters built a configurable Python solver in 2023 with BFS, Dijkstra, A* and IDA*, a transposition table, simple and freeze deadlock detection, and both a graphical and a command-line interface [19]. A* performed best, every added enhancement solved more levels, and deadlock checks reduced the speed per node. The limitations noted were self-made test sets, the speed of Python and weak results on harder sets. In 2024, Lehnert et al. trained transformer models on A* search traces and obtained planners that solve Sokoban with fewer search steps than the A* they learned from [20]. These models are large and need much training data, and their behaviour cannot be read as rules.

2.1.2 Discussion of the Literature

The surveyed work falls into four groups. The complexity results [1], [2] show that Sokoban is intractable in general, so any useful solver has to rely on heuristics and pruning. The University of Alberta group [3], [4], [12], [17] showed that general search breaks down on Sokoban unless it is given domain knowledge, above all about deadlocks, and their strongest solvers combine many such enhancements. A third group [8], [14] uses pattern databases [13] to obtain solutions that are provably the shortest. The most recent group [5], [15], [18], [20] tries to learn this knowledge instead of writing it by hand.

The student projects of Venkatesan et al. [16] and Peters [19] confirm the main design choices of this project: A* is the strongest of the classical algorithms on Sokoban, a matching heuristic improves on plain Manhattan distance, and freeze deadlock detection is useful. Both projects also leave gaps. Deadlock detection was partial or left for future work, results were measured on private level sets without a common baseline, and the solvers ran as desktop programs in which the search could not be watched or compared side by side.

2.1.3 Existing System and its Security Approaches

Sokoban is currently found in three kinds of software. Puzzle games on desktop, mobile and the web allow play, usually with undo, but they have no solver, any hints they offer are not based on search, and they never explain why a position is lost. Research solvers such as those described in [4], [8], [17] are command-line programs that print a solution. They are very strong, but their search cannot be observed, and comparing two techniques requires rebuilding or reconfiguring the program. General planning or puzzle libraries can describe Sokoban as a state-space problem, but they do not know its deadlock patterns, so they spend most of their time on dead positions. The Breadth-First Search results in Chapter 4 show how quickly this becomes a problem.

The security of these systems follows mostly from how they are built. Native games and solvers require the user to download and run an executable, often unsigned, with the user's full permissions. Web games that run on a server move the risk to the server: shared level names can carry stored cross-site scripting, and a deliberately hard level sent to a server-side solver can keep it busy, which is a denial of service. General libraries usually leave input checking and resource limits to the application that uses them. Table 2.1 compares the existing systems with the proposed system on these points.

Table 2.1 Existing Systems Compared with the Proposed System

Aspect	Casual games	Research solvers	Proposed system
Solver and hints	None or scripted	Strong solver, no hints	Four optimal solvers, live hints
Explains dead positions	No	Internally only	Names the rule (corner, wall-line, dead square, freeze)
Comparable metrics	No	Per run, per program	Same level, same budget, four algorithms side by side
Delivery	Install app / web page	Compile or download binary	Static web page, nothing to install
Execution trust model	Native code or server	Native code, full user rights	Browser sandbox (WebAssembly), no server
Input validation	Varies	Usually trusts the file	Strict XSB parser with readable errors
Resource limits	Not applicable	Usually unbounded	Node and time budgets on every search
Data collected	Often analytics, accounts	None	None; preferences stay in the browser

In short, existing systems either search blindly and slow down badly as crates are added, or know too little about deadlocks and waste effort on lost positions. None of them gives a common baseline for measuring each technique or shows the search to a learner, and all of them ask the user to trust native code or a remote server.

2.2 RESEARCH GAPS

The review points to six gaps. Each gap is listed below together with the way the proposed system deals with it.

- **G1, no step-by-step comparison.** Published solvers add many improvements at once, so the effect of each one cannot be separated. The proposed system uses a ladder of four solvers in which each step adds one idea, and all of them run on the same levels with the same limits.

- **G2, incomplete or unexplained deadlock handling.** Student solvers usually check only corners, or leave deadlocks for later [16]. The proposed system implements four rules, precomputes them wherever possible, tests them against exhaustive search and records which rule removed each push.
- **G3, the search cannot be seen.** Existing tools print the answer and nothing else. The solvers in this project can be paused, so the page can draw the position being examined and run four searches side by side.
- **G4, pushes and moves are mixed up.** Many reports call a solution optimal without stating the cost measure. Here the number of pushes is the cost, optimality is proved for that cost, and moves are reported separately.
- **G5, no limits and a trust model based on installing software.** Every search in the proposed system has node and time limits, and the unchanged Python solver runs inside the browser sandbox through WebAssembly [21].
- **G6, fixed sets of levels.** A level editor that checks its input and a generator whose levels are always solvable and are graded by the solver [22] are included.

2.3 PROPOSED SYSTEM

2.3.1 Overview of the Proposed System

The proposed system, Sokoban Search Lab, is a playable Sokoban in which the AI can solve each level in four different ways and show what each extra idea is worth. Figure 2.1 shows the block diagram of the system. A level comes from the built-in library, the editor or the generator. The engine parses it, checks that it is valid and builds the push-level state space. Everything that depends only on the level, such as the dead-square map, the push distances and the nearest-goal table, is computed once at this stage. The pausable search engine then runs one of the four algorithms, and its result (the optimal pushes, the full move string and all counters) feeds the five parts of the interface shown at the bottom of the figure: Play, Hint, Solve, Race and Lab.

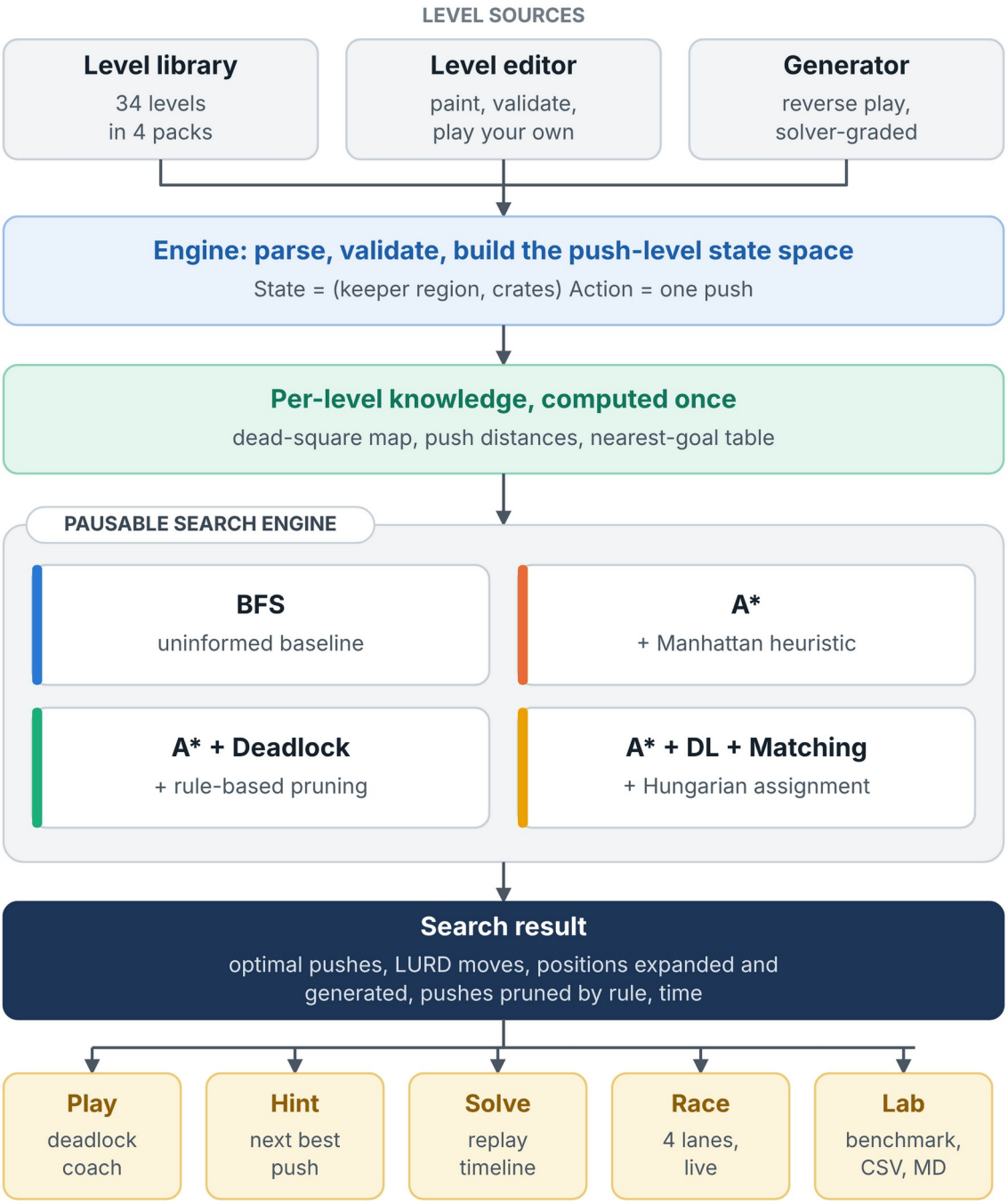
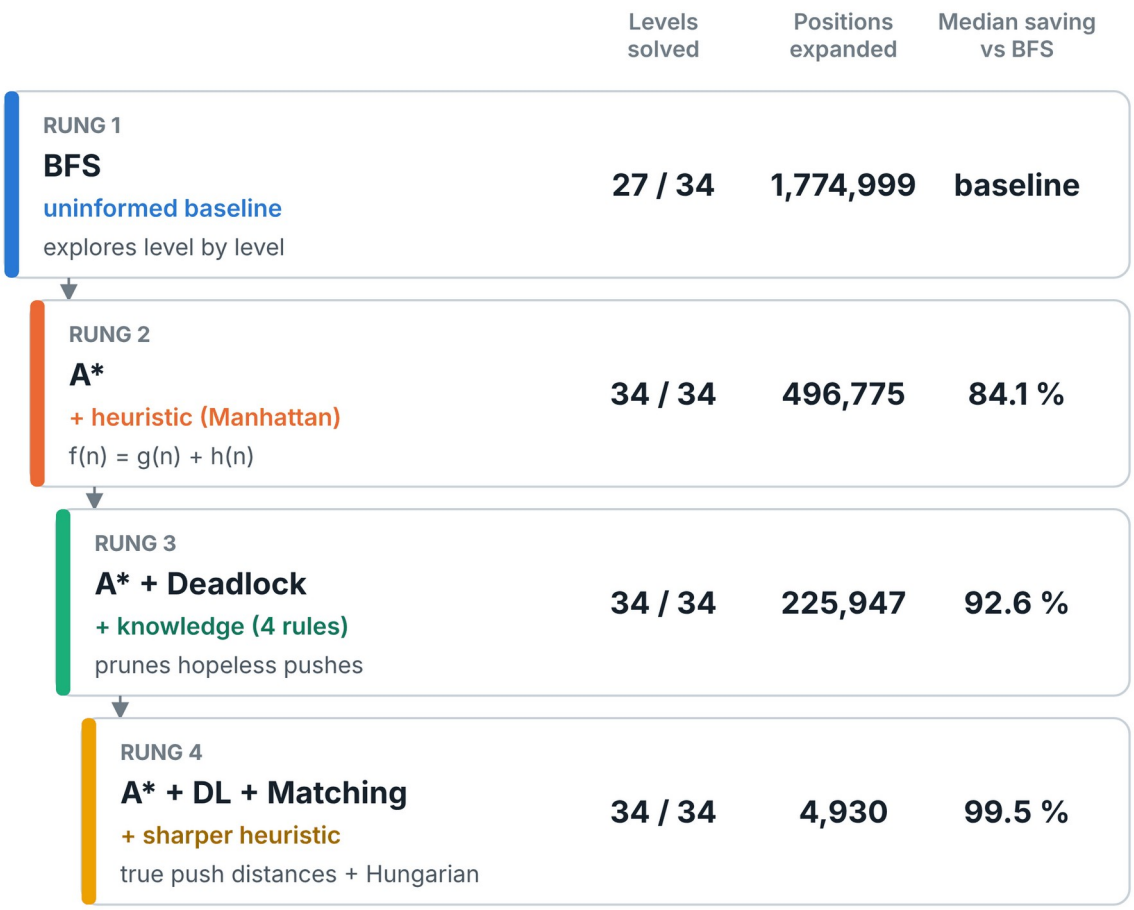


Figure 2.1 Block Diagram of the Proposed System

The central idea of the design is the ladder of four algorithms shown in Figure 2.2. Each rung of the ladder finds the same optimal number of pushes, so whenever two rungs differ in a table, the difference is exactly the value of the extra idea. For each rung, the figure gives the idea it adds and its totals over all 34 levels from Section 4.2.2: the number of levels solved, the positions expanded and the median saving compared with Breadth-First Search.



Each rung adds exactly one idea, so the gap between two rungs is the value of that idea.

Figure 2.2 The Four-Rung Algorithm Ladder with Measured Totals

2.3.2 Objectives of the Proposed System

- Find a solution with the fewest possible pushes for every solvable level within the limits, and declare a level unsolvable only after the whole search space has been checked.
- Reduce the search with admissible heuristics and correct pruning, without giving up optimality.
- Measure what each technique contributes under the same conditions, and allow the results to be exported as CSV or Markdown.
- Show the search while it runs: draw the position being expanded, race the algorithms, and replay or scrub through solutions.
- Help the player with hints that point to the next best push and a coach that names the deadlock rule just broken.

- Run on any device with a browser, using one implementation for the browser, the command line and the tests.

2.3.3 Advantages of the Proposed System

- The gains are large and verified. On the same 34 levels, total positions expanded drop from 1,774,999 to 4,930 and total time from 160.8 s to 0.56 s, while the push counts stay the same.
- The pruning can be explained. Each removed push is credited to a named rule, and the same rules shade the dead squares on the board and warn the player when a level becomes unwinnable.
- There is only one copy of the algorithms. Through Pyodide, the browser runs the same Python files that the tests check, so there is no JavaScript version that could drift out of step.
- Much of the security comes from the design itself. There is no server, no login and no data collection, input is validated, every search is limited, and the page can call only a fixed list of public methods.
- The system is easy to extend. A new level is a text file in a pack folder, and a new algorithm only has to be added to the ladder for the race, the benchmark and the reports to include it.
- It costs nothing to run: standard library only, no GPU and free static hosting, on any recent laptop or phone browser.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The system is organised in layers, as shown in Figure 3.1. GitHub Pages delivers the static files over HTTPS to the presentation layer, which is a single web page with five views written in HTML5, CSS3 and plain JavaScript. `render.js` draws the board on a canvas, `app.js` handles input, the timeline and the animation loop, and `localStorage` keeps a few preferences. Below it is the runtime layer, Pyodide, which is CPython compiled to WebAssembly [21]; the page loads it together with a JSON file holding the five Python modules and the 34 levels. JavaScript and Python communicate only through the interface bridge, `interface/bridge.py`: the page calls `call(method, json)` and receives JSON back. Under the bridge are the solvers, the analytics and the puzzle library, and at the bottom is the engine, which holds the rules of the game. In the figure, solid arrows are requests and dashed arrows are replies, and every arrow runs straight between two neighbouring layers. The box at the bottom shows that the command-line benchmark, the unit tests and the site builder use the same core directly, without the browser.

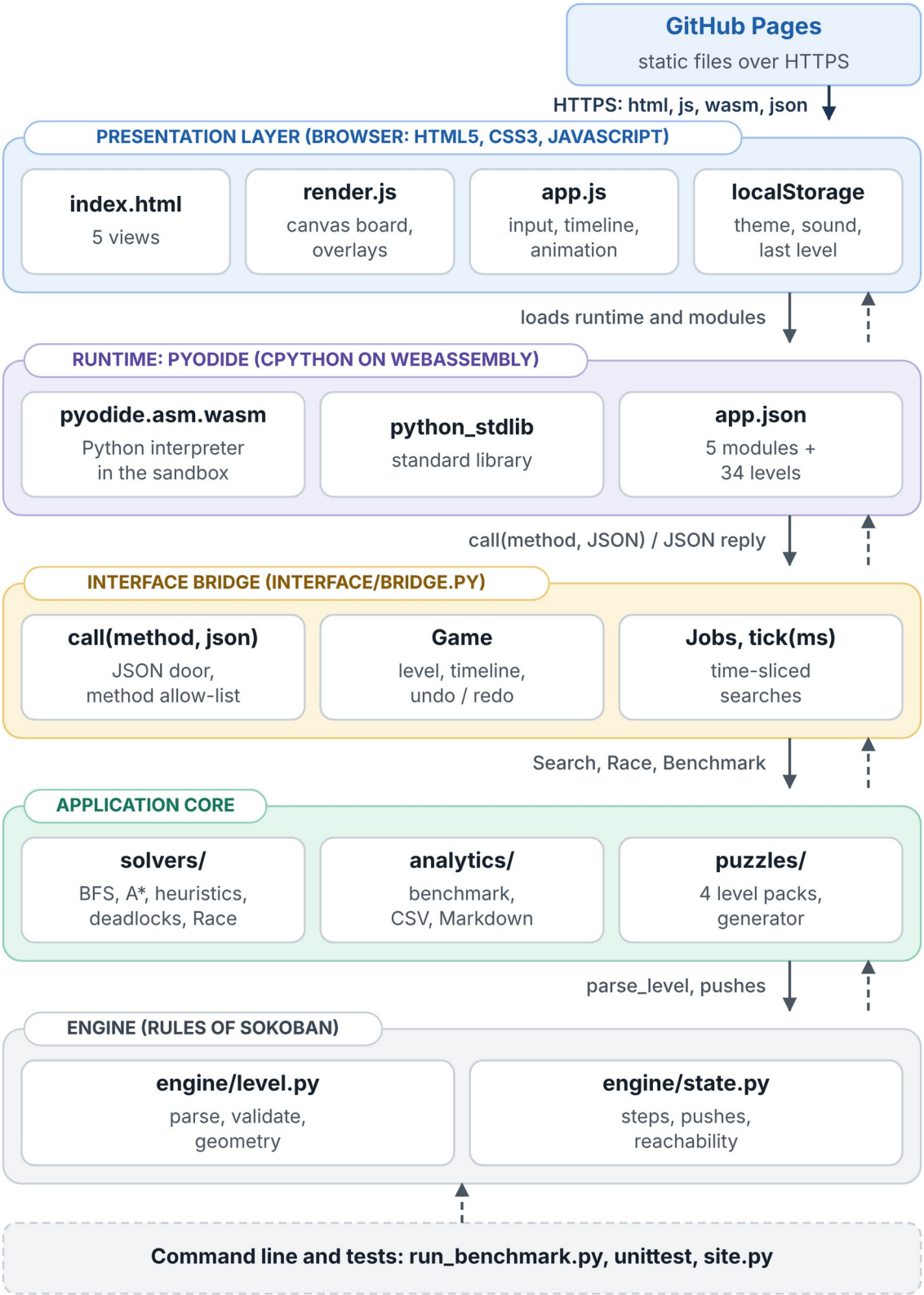


Figure 3.1 Layered System Architecture

A long search must not freeze the page. Every solver is therefore written as a Python generator that pauses after every 32 expansions, and a `Search` object runs it for a given slice of time. On each animation frame the page calls `tick(10)`, and the bridge divides those 10

milliseconds among the running jobs, which may be a solve, a hint, a race or a benchmark. The interface stays responsive, and four algorithms can run side by side in a race without threads. Table 3.1 lists the hardware needed to run the system.

Table 3.1 Hardware Requirements

Component	Minimum	Recommended / remarks
Processor	Any 64-bit dual-core CPU (Intel Core i3 / AMD Ryzen 3 class)	Quad-core; the solver uses one core
Memory (RAM)	4 GB	8 GB; a browser tab with the runtime usually takes a few hundred MB
Storage	About 25 MB for the site and runtime	About 50 MB for the repository with history
Display	1280 × 720	1920 × 1080; the layout also adapts to phones
GPU	Not required	Classical search only, no machine learning
Network	Only for the first load of the hosted site (about 21 MB)	Works fully offline when served locally

Table 3.2 lists the software used. The solver needs only the Python standard library, so nothing has to be installed.

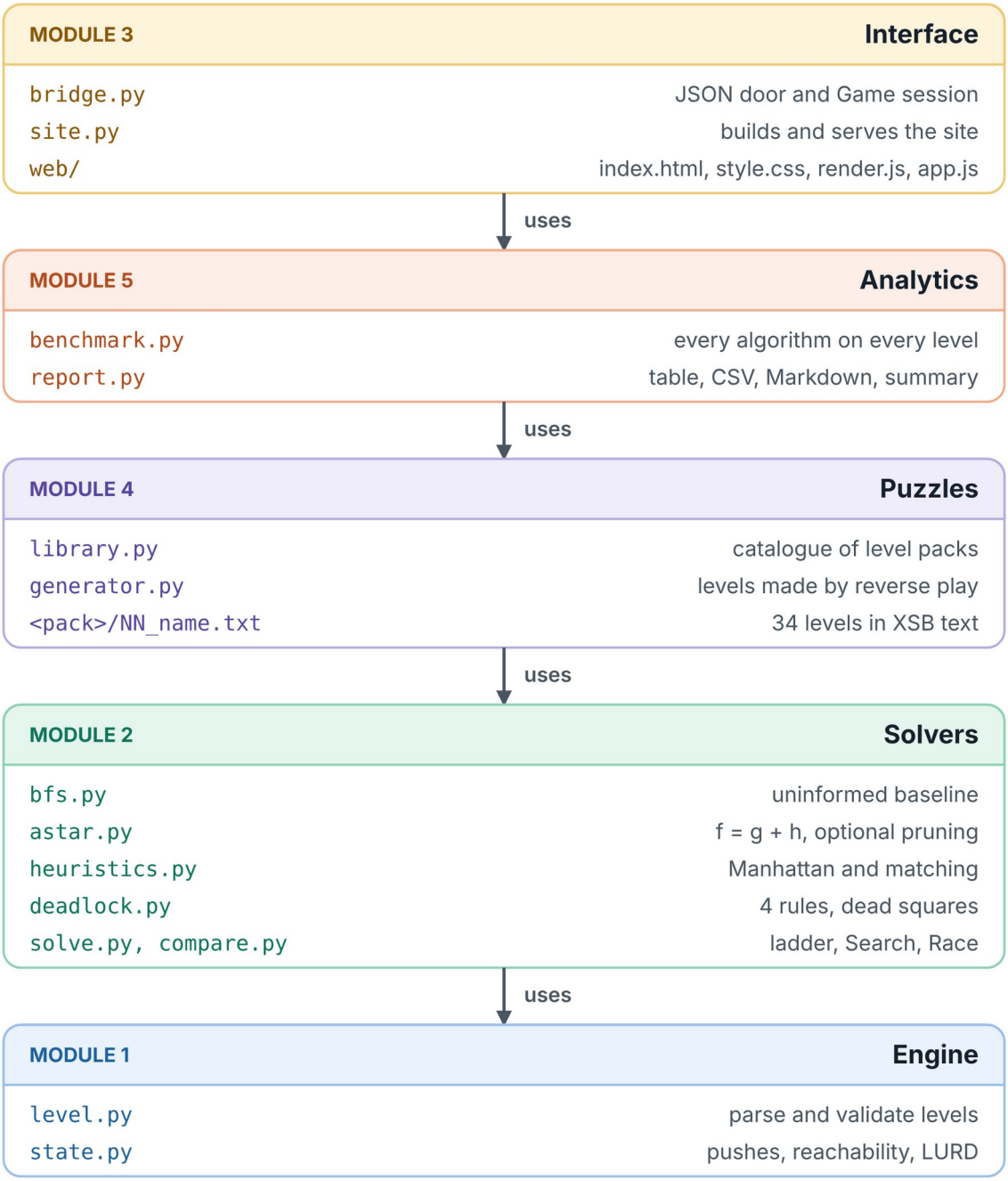
Table 3.2 Software Requirements

Category	Software	Purpose
Language	Python 3.9 or newer (tested on 3.11 and 3.13)	Engine, solvers, puzzles, analytics, bridge
Libraries	Python standard library only: heapq, collections, itertools, time, json, csv, random, pathlib	Priority queue, queues, records, timing, serialisation, reports
Browser runtime	Pyodide 314.0.7 (Python 3.14.2 on WebAssembly)	Runs the unchanged Python modules in the page
Front end	HTML5, CSS3, ES6 JavaScript, Canvas 2D, Web Audio	Interface, board drawing, synthesised sound
Browser	Any current Chrome, Edge, Firefox or Safari	Needs WebAssembly support

Category	Software	Purpose
Tools	Git and GitHub, VS Code or PyCharm, unittest	Version control, development, testing
Hosting	GitHub Pages (static, HTTPS); any static host	Public deployment without server code

3.2 MODULE DESIGN

The code is divided into five modules whose dependencies point in one direction only. Figure 3.2 stacks the modules in dependency order and lists the files in each one: the interface uses the analytics, the analytics use the puzzles, the puzzles use the solvers, and the solvers use the engine, which imports nothing else. Since a module may use only the modules below it, the dependency graph has no cycles, and the same solver code can run from the command line, in the tests and in the browser.



Each module may use only the modules below it, so the dependency graph has no cycles.

Figure 3.2 Module Design and Dependency Order

Table 3.3 summarises the responsibilities of each module.

Table 3.3 Module Description

Module	Files	Responsibilities
1. Engine	level.py, state.py	Parse and validate XSB text; immutable `Level` geometry with integer cells and a neighbour table; `State(player, boxes)`; single steps, legal pushes, keeper reachability, LURD move strings
2. Solvers	bfs.py, astar.py, heuristics.py, deadlock.py, solve.py, compare.py	Uninformed and informed search as pausable generators; Manhattan and matching heuristics; Hungarian algorithm; dead-square map and freeze detection; the algorithm ladder, `Search`, `Race`, comparison with BFS
3. Interface	bridge.py, site.py, web/	JSON door and `Game` session (timeline, undo/redo, jobs, hints, coach); static-site build and local server; the five-view web page
4. Puzzles	library.py, generator.py, 34 level files	Catalogue of packs read from folders; level generator using reverse play and solver grading
5. Analytics	benchmark.py, report.py	Pausable benchmark of every algorithm on every level; text table, CSV, Markdown and per-algorithm summary

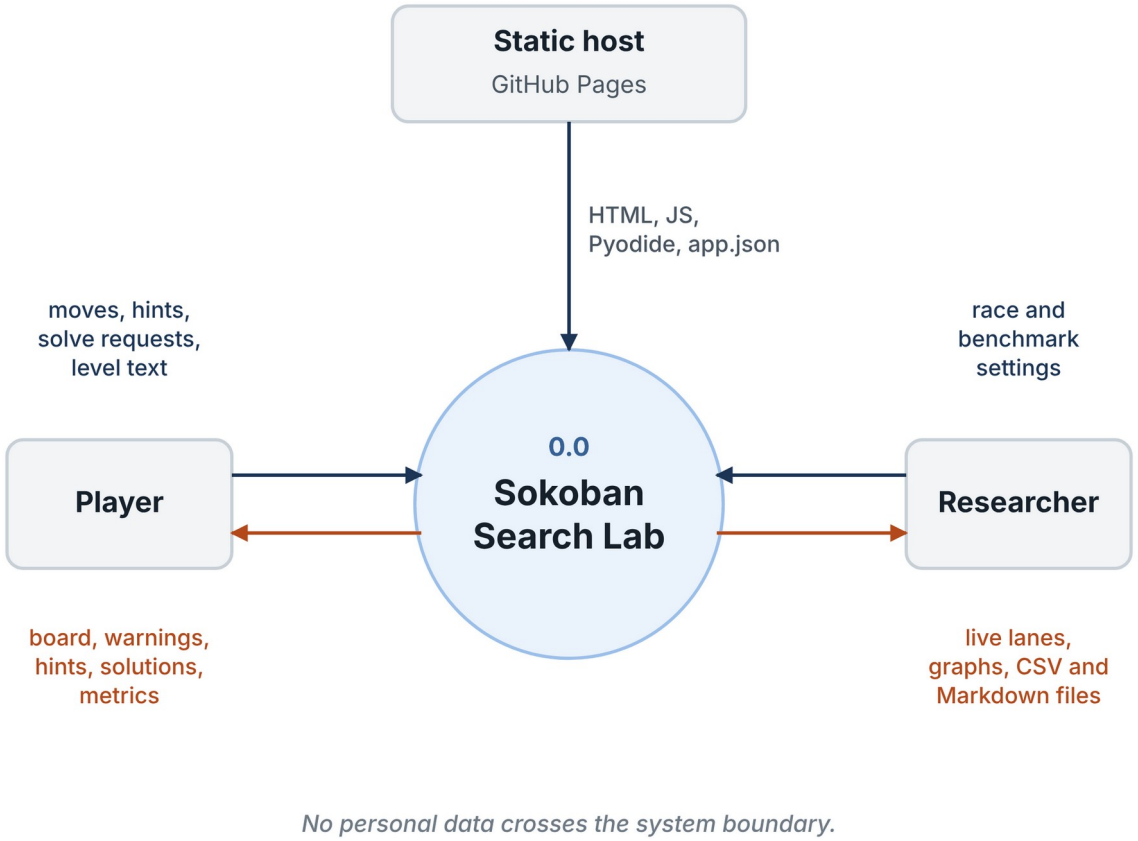
In the engine, each cell is stored as one integer, `y · width + x`, so moving one step is a single addition: ±1 horizontally and ±width vertically. Every level is surrounded by walls, so every floor cell has four neighbours inside the grid and the search never needs bounds checks. `reachable` flood-fills the keeper's region, and `legal_pushes` lists every (crate, direction) pair for which the keeper can get behind the crate and the square beyond it is free. `pushes_to_moves` converts a solution into the standard LURD notation, with lower case for walking and upper case for pushing.

The two search functions, `bfs` and `astar`, share the same state handling and the same code for tracing the path back. `astar` accepts an optional `Deadlocks` object, so A* with deadlock pruning is the same search with an extra filter rather than a separate algorithm. `solve.py` defines the ladder and the `Search` class with its node and time limits, and `compare.py` provides `Race` and the comparison with Breadth-First Search used by both the command line and the browser.

In the interface module, `Game` keeps a single timeline of positions that holds both the player's moves and the AI's solution, so undo, redo, replay and scrubbing are all movements of one cursor along that timeline. `Game` also computes the AI optimum for the level (shown to the player as “AI best”), the trails for the path comparison overlay, and the deadlock coach message for the current position. The puzzles and analytics modules are described in Chapter 4.

3.3 DATA FLOW / UML DIAGRAMS

Figure 3.3 is the context diagram (data flow diagram, level 0). It treats the whole system as one process that exchanges data with three external entities. The player sends moves, hint and solve requests and level text, and receives the board, deadlock warnings, hints, solutions and metrics. The researcher sends race and benchmark settings and receives live lanes, graphs and CSV or Markdown files. The static host delivers the page and the Python bundle. No personal data crosses this boundary.



No personal data crosses the system boundary.

Figure 3.3 Data Flow Diagram – Level 0 (Context Diagram)

Figure 3.4 refines the context diagram into six processes and five data stores. A level is loaded from the level library and validated (1.0), and its knowledge tables are computed

and stored (2.0). The search (3.0) reads these tables and is started by the play session (4.0), which sends a request and receives the solution and keeps every position in the session timeline. The rendering process (5.0) draws the view for the player using the stored preferences, and the comparison process (6.0) runs races and benchmarks for the researcher and saves the benchmark rows. Every flow in the figure is a straight horizontal or vertical arrow, and no two flows cross.

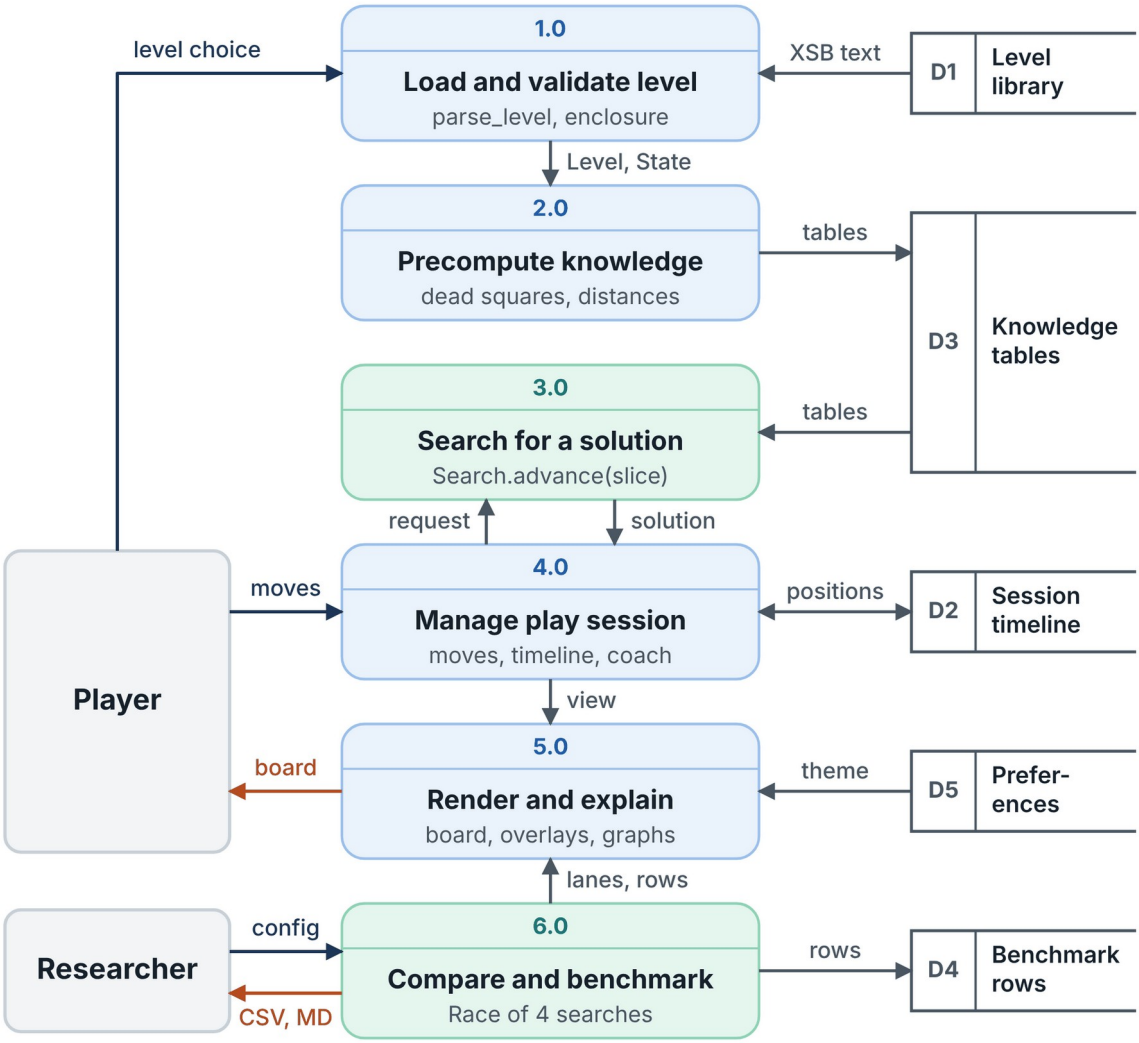


Figure 3.4 Data Flow Diagram – Level 1

Figure 3.5 is the use case diagram, with two actors. The player plays levels, undoes moves, requests hints, asks the AI to solve with a chosen algorithm, replays the solution, shows the dead-square and search-trace overlays, compares their own path with the AI's path, and builds or generates levels. The researcher races or benchmarks the algorithms and exports the results. Requesting a hint, solving, and racing or benchmarking all include the “Run search” use case, which in turn includes deadlock detection.

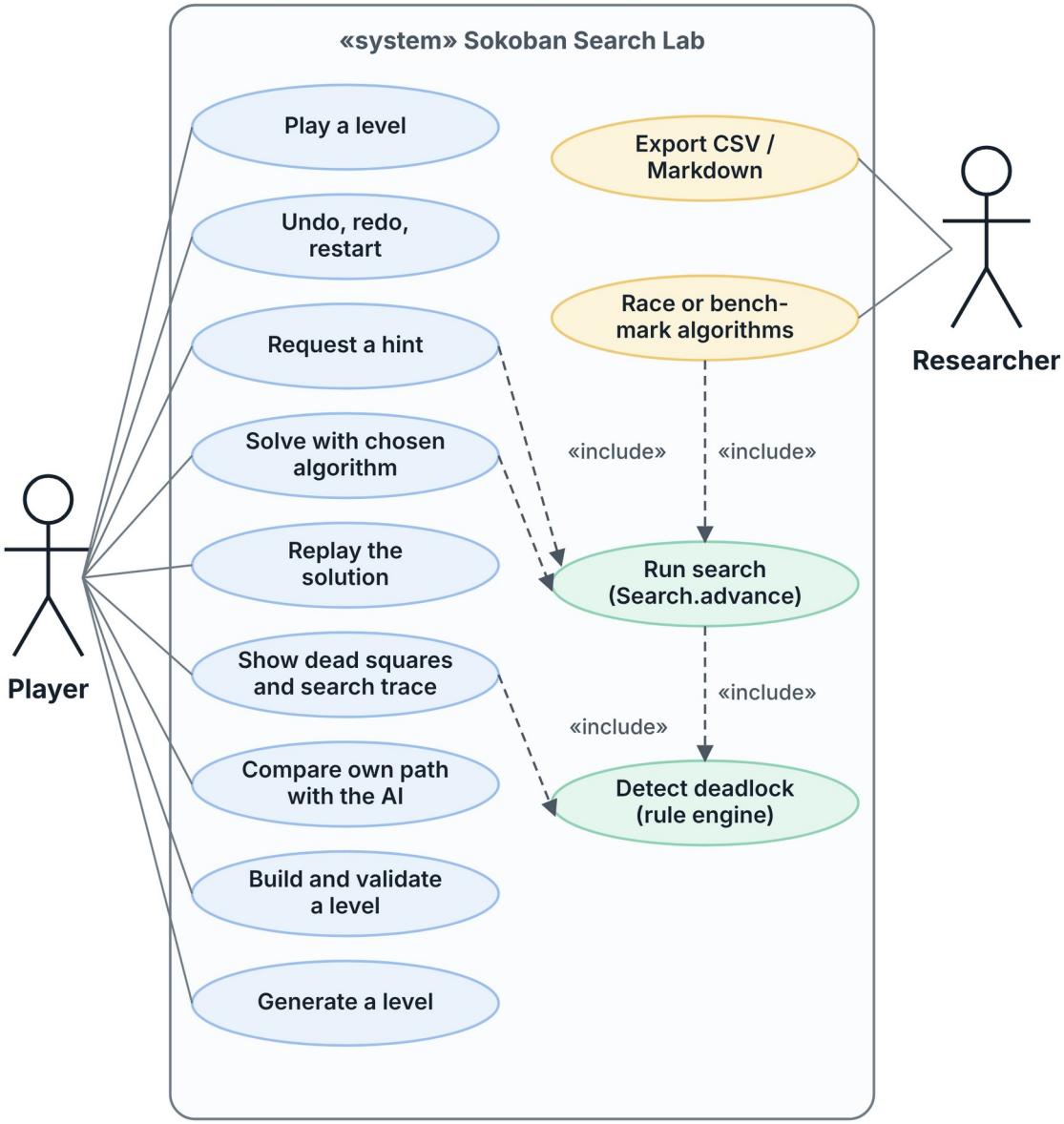


Figure 3.5 Use Case Diagram

Figure 3.6 is the activity diagram of the main algorithm, A* with deadlock pruning. After the tables are built and the start position is checked, each round takes the position with the lowest f, skips it if it is a stale heap entry, and applies the goal test. Testing the goal at expansion time is what keeps the result optimal. Otherwise the keeper's region is filled and every legal push is examined in the inner loop on the right: a push is skipped if it leads to a closed or no cheaper state, pruned if a deadlock rule fires or the heuristic shows the position cannot be solved, and added to the frontier in all other cases. The search stops with a plan, with a report that the level is unsolvable, or when the node or time limit is reached.

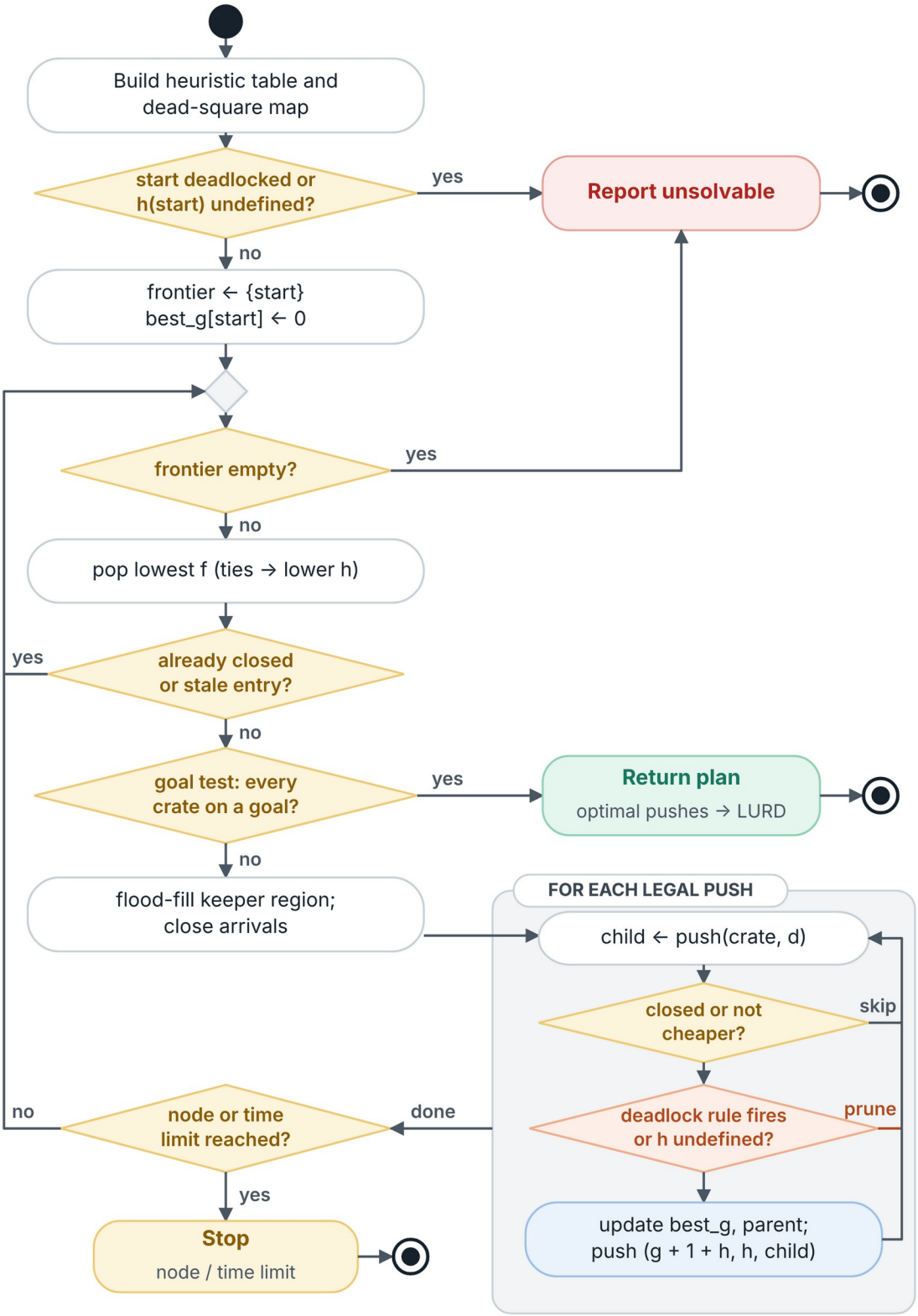


Figure 3.6 Activity Diagram of A* Search with Deadlock Pruning

Figure 3.7 is the sequence diagram of a single Solve request. The click reaches `app.js`, which calls the bridge, and `Game` creates a `Search`. Inside the loop fragment, every animation frame sends a `tick` that runs the search for a short slice of time; the progress

returns as JSON, and `render.js` draws the position being examined. In the optional fragment at the bottom, the finished solution is appended to the timeline and played back to the player.

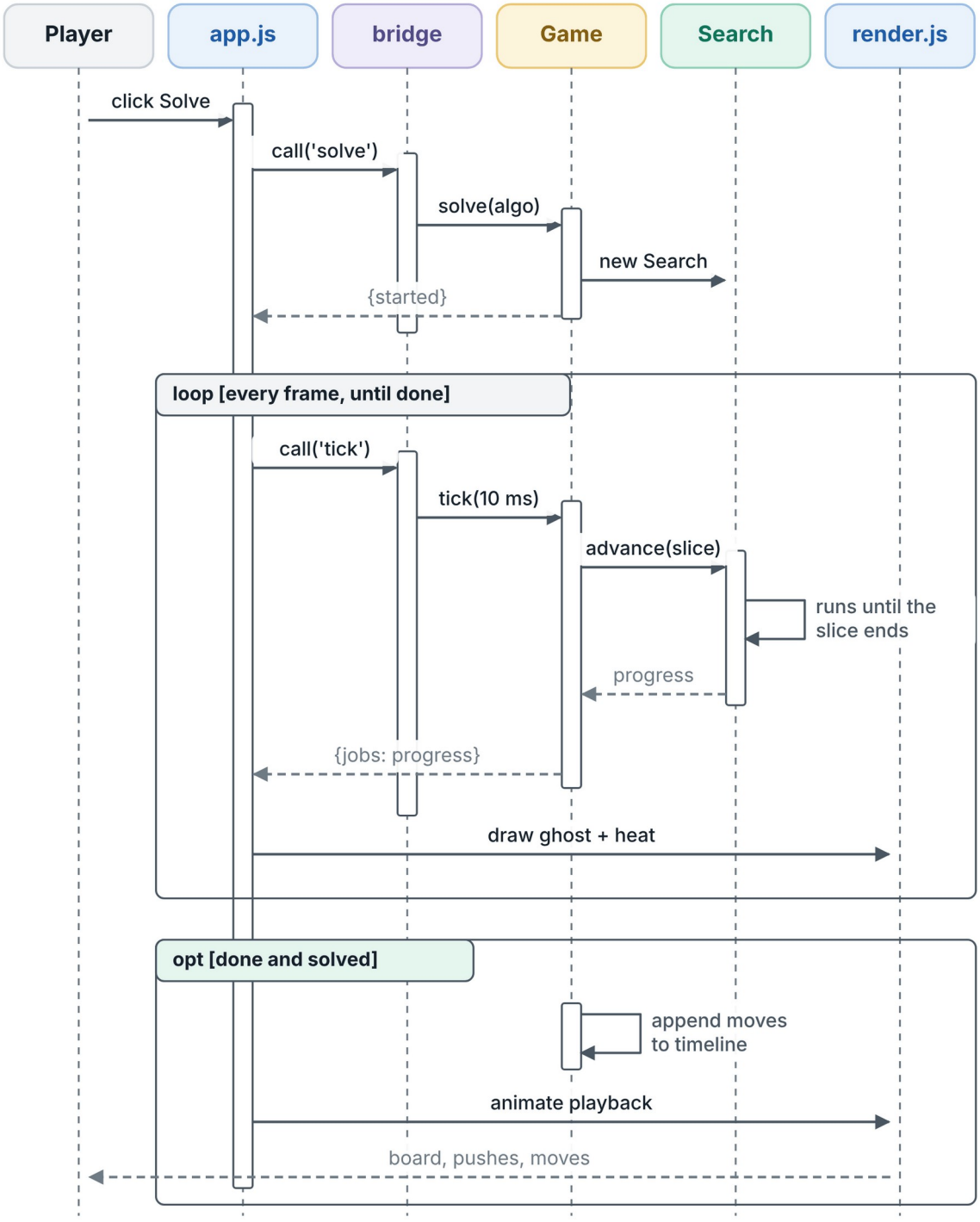


Figure 3.7 Sequence Diagram of a Solve Request

Figure 3.8 is the class diagram of the main types. `Game` holds the running jobs, `Race` holds four `Search` objects, and `Benchmark` holds one `Race` per level. `Search` runs the

module-level search functions on a `Level` and a `State`, and `Deadlocks` stores the precomputed dead-square map of one level. Each class box lists its attributes above its operations.

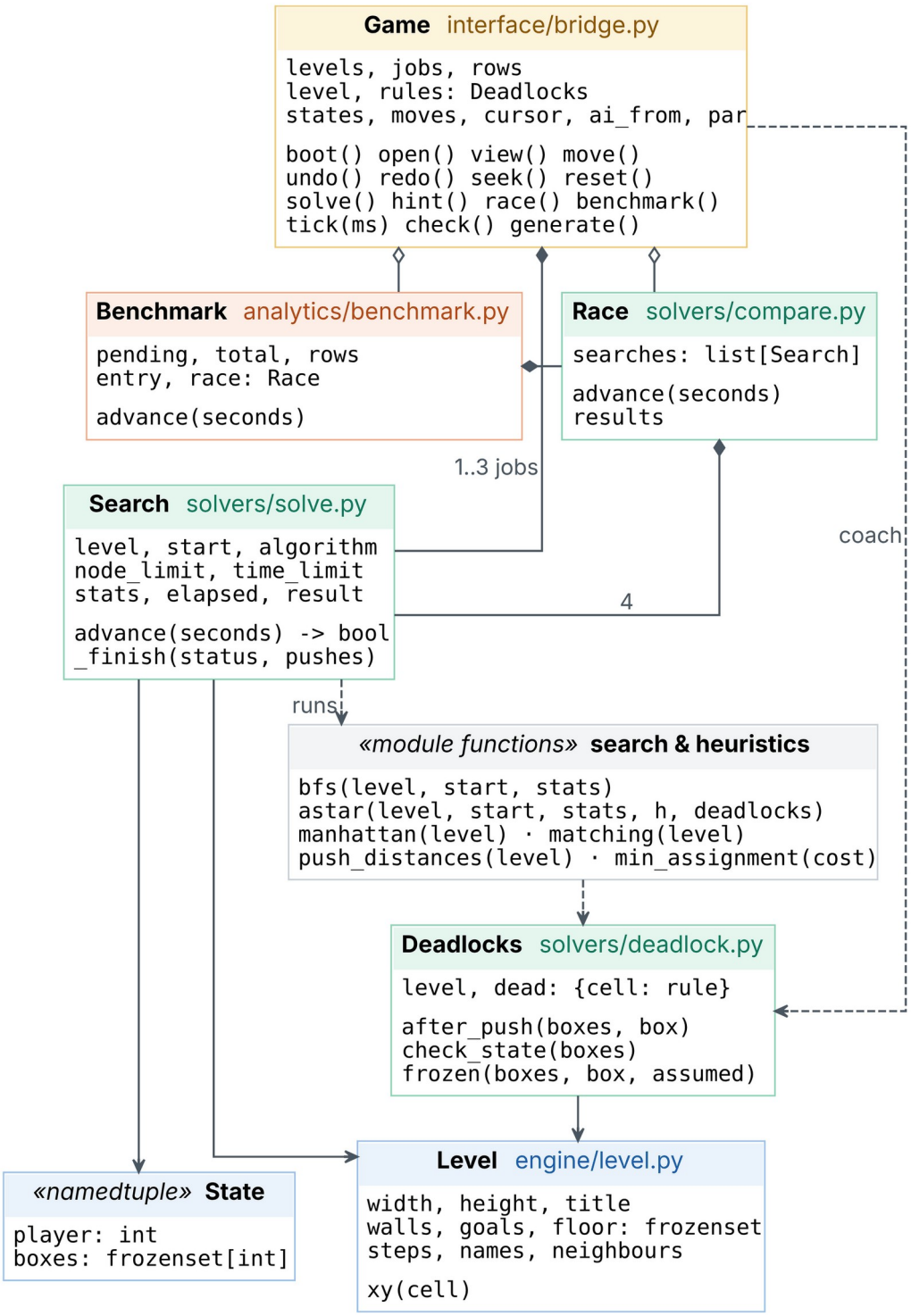


Figure 3.8 Class Diagram of the Core Types

3.4 THREAT MODEL AND ATTACK SURFACE

The threat model follows the STRIDE method [10]. All processing takes place on the client, and the server only serves static files. Figure 3.9 marks three trust boundaries. At the top left is the supply chain, with the GitHub repository, the vendored Pyodide runtime and the site test. At the top right is the public internet, with the GitHub Pages CDN and the network path. The large boundary below is the browser sandbox on the user's device, which contains the player, the page scripts, the bridge, the Python core, local storage, the clipboard and the level editor. The red badges T1 to T8 mark the points where the eight threats enter.

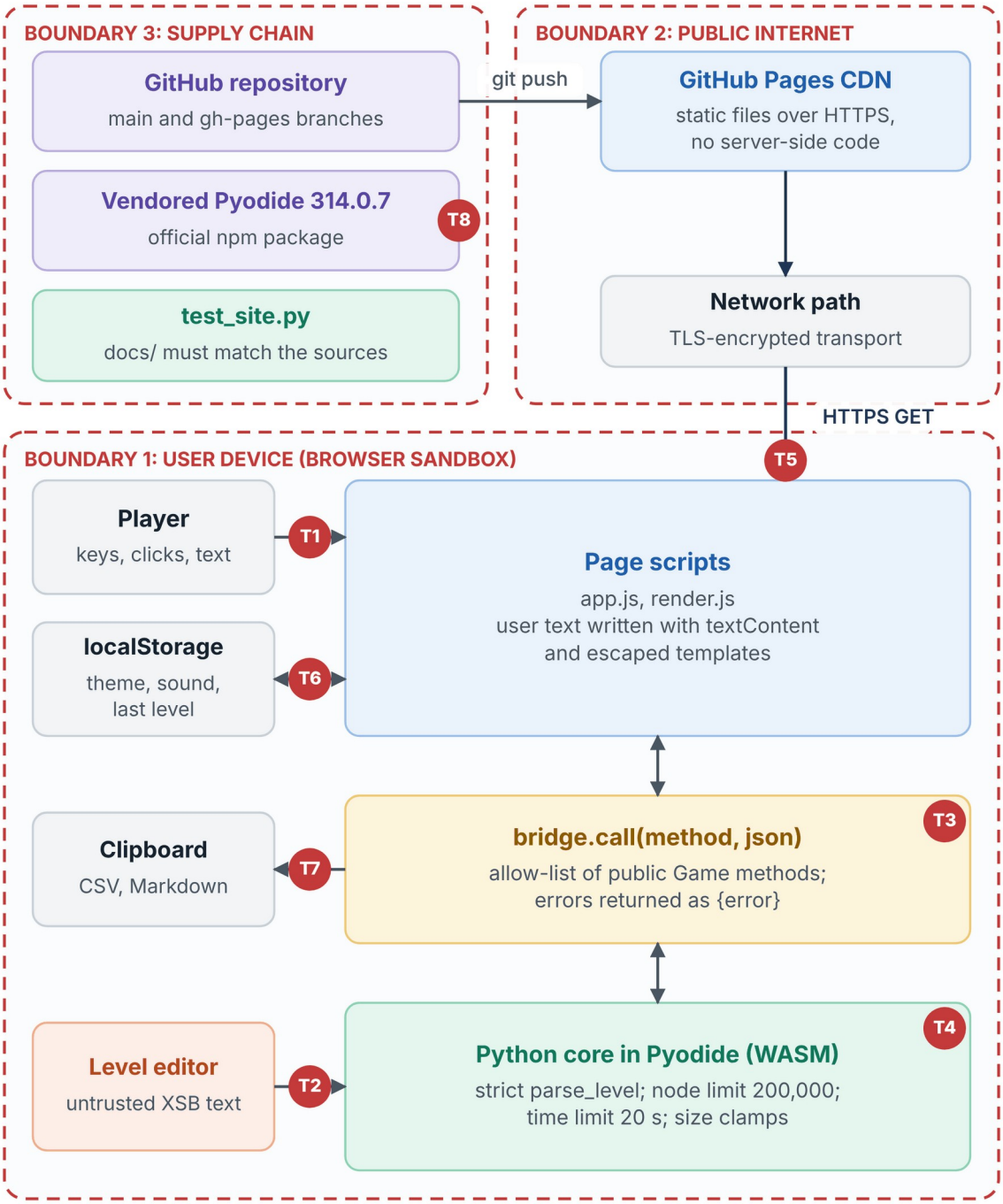


Figure 3.9 Threat Model: Trust Boundaries and Attack Surface

Table 3.4 lists the assets that need protection and the security objective for each one.

Table 3.4 Assets and Security Objectives

Asset	Security objective	Why it matters
Delivered code (HTML, JS, WASM runtime, Python bundle)	Integrity	Tampered code would run with the page's rights in every visitor's browser
Solver correctness and benchmark results	Integrity	The claims in this report depend on correct optimal pushes and honest counts
Responsiveness of the user's device	Availability	A crafted level must not hang the tab or use up memory
The user's browser context (DOM, storage)	Integrity, confidentiality	Injected script could act on the page or read storage
Source repository and release branch	Integrity	Decides what every visitor receives

Table 3.5 analyses each of the eight threats marked in Figure 3.9, with its STRIDE category, entry point, risk level and mitigation.

Table 3.5 STRIDE Threat Analysis

ID	Threat (STRIDE)	Entry point	Risk	Mitigation in the system
T1	Script injection through a level title or note (Tampering / Elevation)	Level text, editor, DOM	Medium	Titles and notes written with `textContent`; templates escape with `esc()`; raw user text never goes into `innerHTML`
T2	Malformed or hostile level text (Tampering)	Editor, level text box	Medium	Strict parser: symbol allow-list, one keeper, crates = goals, enclosure and reachability checks, readable errors
T3	Calling internal methods through the bridge (Elevation of privilege)	`bridge.call(method, json)`	Low	Only public callable `Game` methods; underscore names refused; arguments parsed as JSON keywords; errors returned, never raised

ID	Threat (STRIDE)	Entry point	Risk	Mitigation in the system
T4	Algorithmic-complexity denial of service (DoS)	Solve, hint, race, benchmark, generator	High	Node limit 200,000 and time limit 20 s per search; time-sliced execution; generator size clamps; benchmark budget ≤ 30 s
T5	Tampering with files in transit (Tampering / Spoofing)	Network path	Low	HTTPS delivery by GitHub Pages; static files only; no mixed content
T6	Edited localStorage values (Tampering)	Browser storage	Low	Only theme, sound and last level stored; reads wrapped in try/catch; an unknown level id is rejected by the bridge
T7	Formula injection in exported CSV (Tampering)	Copy as CSV / Markdown	Low	Exports contain only catalogue level titles and numbers; user-made titles are never benchmarked
T8	Supply-chain compromise of runtime or repository (Tampering)	Vendored Pyodide, Git branches	Medium	Runtime taken unmodified from the official npm package and pinned; reproducible build; a test fails if `docs/` differs from the sources

Repudiation and information disclosure mostly do not apply to this system. There are no accounts, no transactions, no server logs and no personal data. The Git history records every change to the code, and no data about the user leaves the browser.

3.5 SECURITY CONTROLS

Table 3.6 lists the implemented controls and the threats that each one handles. The controls are grouped into preventive controls, which stop an attack, limiting controls, which reduce its effect, and detective controls, which reveal a problem before release.

Table 3.6 Implemented Security Controls

Control	Type	Implementation	Threats
C1 Input validation	Preventive	`engine/level.py`: symbol allow-list, structural rules, flood-fill enclosure and reachability; `LevelError` with a reason the user can read	T2

Control	Type	Implementation	Threats
C2 Output encoding	Preventive	`app.js`: `textContent` for user text, `esc()` for templated values	T1
C3 Interface allow-list	Preventive	`bridge.call`: rejects private and non-callable names; JSON in, JSON out; exceptions turned into `{error}`	T3
C4 Resource budgets	Limiting	`Search(node_limit, time_limit)`, time-sliced `advance`; clamps in `Game.generate` and `Game.benchmark`	T4
C5 Sandboxed execution	Limiting	Python in WebAssembly inside the browser sandbox; no file system, network or `eval` of user input	T1–T4
C6 Transport security	Preventive	HTTPS-only static hosting; no third-party scripts or analytics	T5
C7 Minimal client storage	Preventive	Three preference keys, try/catch around every access	T6
C8 Pinned, reproducible build	Detective	Vendored, pinned runtime; `tests/test_site.py` checks that the published site matches the sources	T8
C9 Automated verification	Detective	59 unit and fuzz tests, including soundness of every deadlock rule	Integrity of results

Three risks remain open and are planned for Phase II. First, `parse_level` does not limit the size of pasted text, so a user could slow down their own browser tab with a very large level; a limit such as 64 × 64 cells would close this gap. Second, GitHub Pages does not allow custom HTTP headers, so a Content Security Policy [23] will be added as a `<meta>` tag that allows only same-origin scripts plus `'wasm-unsafe-eval'` for the runtime. Third, the release branch should be protected so that every change needs a review and signed commits.

CHAPTER 4

SYSTEM DESCRIPTION

4.1 DATA COLLECTION

For a search project, the data are the puzzles themselves. The system uses 34 Sokoban levels stored as plain XSB text files, one file per level and one folder per pack. In XSB, `#` is a wall, a space is floor, `.` is a goal, `$` is a crate, `*` is a crate on a goal, `@` is the keeper and `+` is the keeper standing on a goal. Lines that start with `;` hold the title and a short note shown to the player. A new file placed in a pack folder appears in the game, the benchmark and the tests without any code change. Table 4.1 summarises the four packs.

Table 4.1 Level Corpus by Pack

Pack	Levels	Crates	Floor cells	Optimal pushes	Purpose and source
Training	7	1–3	18–29	2–15	Small levels; one idea per level (pushing, corners, wall lines, freezes, order)
Classic nine	9	1–4	30–94	4–14	The levels from the first review; open rooms
Warehouse	9	2–5	25–53	11–21	Tight rooms from the reverse-play generator, where deadlock knowledge matters
Showdown	9	4–6	34–60	14–21	Hard rooms picked so that all four algorithms behave differently
Total	34	1–6	1,417 cells	2–21	

Figure 4.1 shows one level from each pack, with its number of crates and its optimal number of pushes. The levels grow from a single crate in an open room (Corner Trap) to four crates in tight rooms with many dead squares (Pallet Maze and Tight Turn).

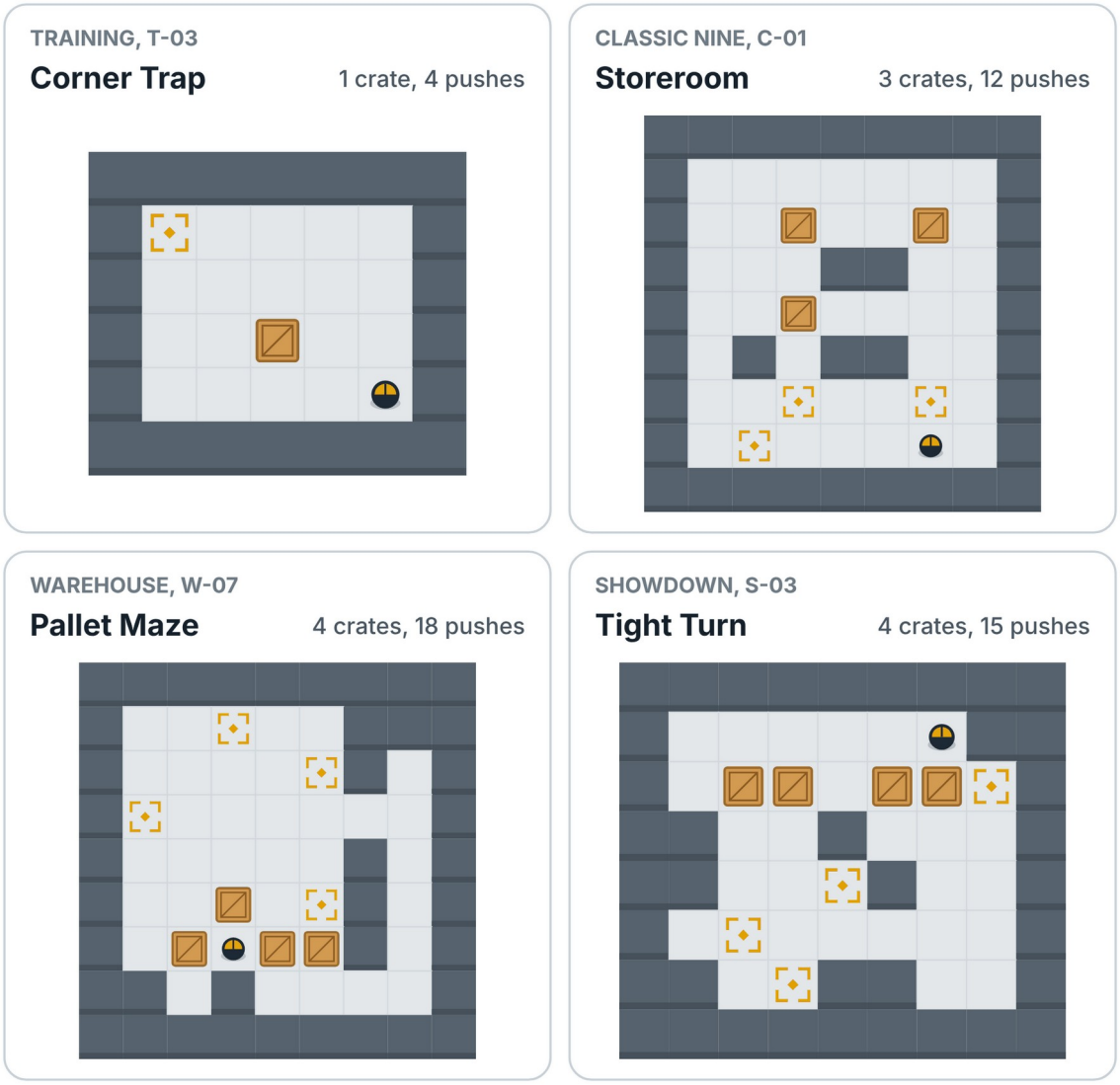
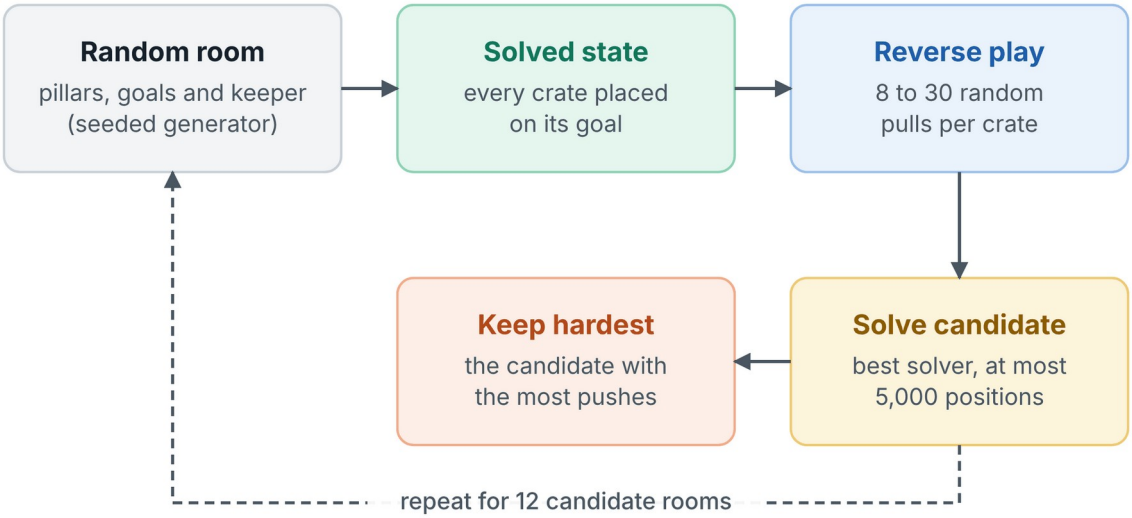


Figure 4.1 Sample Levels from the Four Packs

The Warehouse levels come from the generator in `puzzles/generator.py`, which follows the idea of producing Sokoban levels by playing backwards [22]. Figure 4.2 shows its steps. The generator builds a random room, places every crate on its goal, and lets the keeper pull crates away from the goals for a random number of moves. A pull is the reverse of a push, so pushing everything back always works and every generated level is solvable. Random pulling alone tends to give dull levels, so the generator creates twelve candidate rooms, solves each one with the best solver (limited to 5,000 positions) and keeps the one that needs the most pushes. A seed makes every generated level reproducible.



Every pull reverses a legal push, so every generated level can be solved.

Figure 4.2 Level Generation by Reverse Play and Solver Grading

The second kind of data is the measurements. Every algorithm was run on every level from the same starting position with `python run_benchmark.py --node-limit 200000 --time-limit 20`, on Python 3.13.16 under Linux, using one CPU core. Table 4.2 defines each recorded metric. When Breadth-First Search hits a limit before it finishes, its true cost is higher than the recorded number, so any saving measured against it is a lower bound.

Table 4.2 Metrics Recorded for Every Run

Metric	Definition
Positions expanded	States taken from the frontier and expanded (the search steps); the main measure of effort
Positions generated	Child states added to the frontier
Pushes pruned (by rule)	Pushes removed by a deadlock rule or by the matching heuristic, counted per rule
Solution pushes	Path cost of the solution; the same for every algorithm that finishes, which is the evidence of optimality
Solution moves	Length of the LURD move string including walking; can differ when there are several optimal solutions
Time	Wall-clock time including each algorithm's set-up (tables, dead squares), measured with a high-resolution timer

Metric	Definition
Fewer nodes than BFS	100 × (1 − expanded / expanded by BFS); a lower bound when BFS hit a limit
Status	solved, unsolvable, node_limit or time_limit

4.2 DATA PREPROCESSING

Before the search starts, the level text is converted into a form that the search can use quickly. Figure 4.3 shows the nine steps of this conversion. Steps 1 to 5 validate the input, which is security requirement SR-1. Step 6 builds the geometry used by the search, and steps 7 to 9 compute the level-specific knowledge in advance, so that it costs nothing during the search.

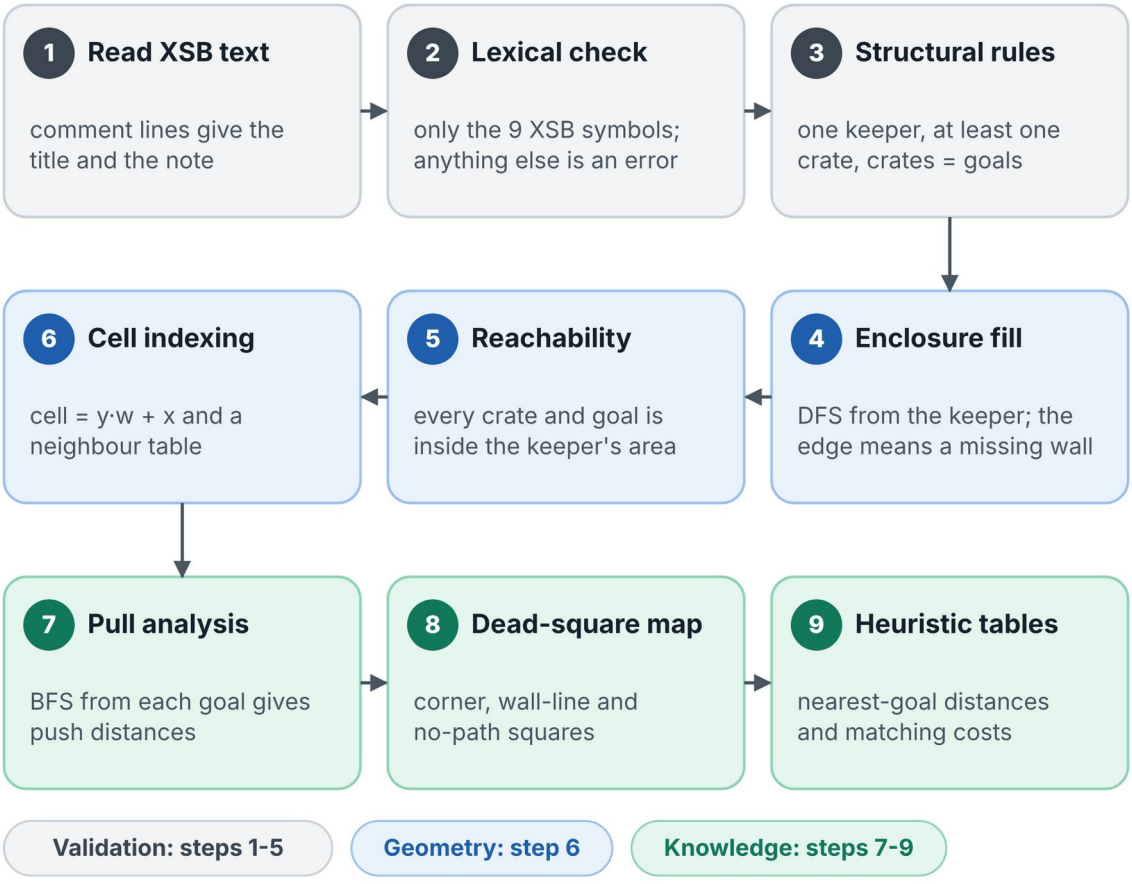


Figure 4.3 Data Preprocessing Pipeline from XSB Text to (Level, State)

Enclosure and reachability are checked with a depth-first flood fill from the keeper, which collects the floor inside the level. If the fill reaches the edge of the grid, a wall is missing and the level is rejected. Any crate or goal outside the filled area is rejected as well. Floor that the keeper can never reach is treated as wall.

The pull analysis runs a breadth-first search from each goal and moves a crate *backwards*. A crate at square c could have come from c − d, provided the keeper had room to stand at c − 2d to push it. This gives the exact number of pushes needed to bring a single crate from any square to that goal, taking walls and the keeper's position into account. A floor square that does not appear in any goal's table is a **dead square**: a crate on it can never reach a goal.

Each dead square is then labelled with the most specific rule that explains it. It is a *corner* if it has a wall on one horizontal side and one vertical side, a *wall-line* square if it lies on a straight stretch of wall that is closed at both ends and has no goal, and a *no-path* square otherwise. Figure 4.4 shows the resulting maps for two levels. Divider contains all three kinds, with a column of no-path squares beside the central wall, while in Pallet Maze 13 of the 41 floor squares are dead.

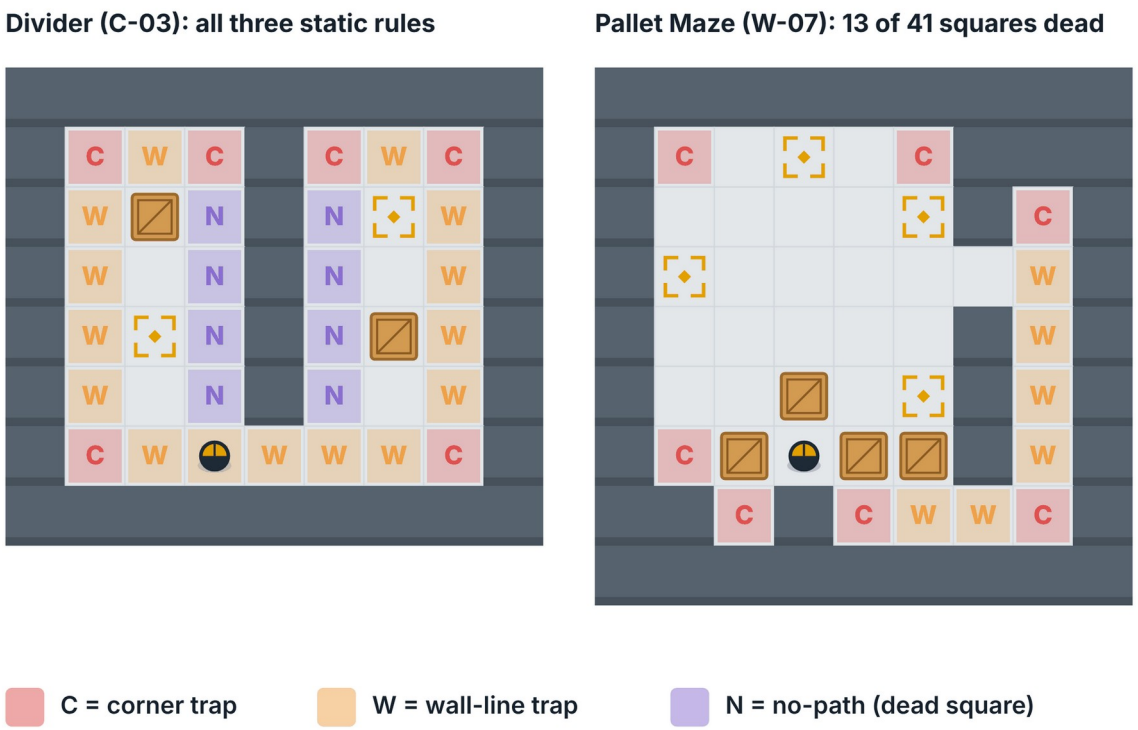


Figure 4.4 Precomputed Dead-Square Maps (C = corner, W = wall-line, N = no-path)

Table 4.3 gives the dead-square counts for all four packs. More than a third of all floor squares, 529 out of 1,417, are dead.

Table 4.3 Dead Squares Precomputed per Pack

Pack	Floor cells	Corner	Wall-line	No-path	Dead total	Share dead
Training	161	28	56	0	84	52.2 %
Classic nine	517	43	117	20	180	34.8 %
Warehouse	350	73	43	0	116	33.1 %
Showdown	389	95	53	1	149	38.3 %
All levels	1,417	239	269	21	529	37.3 %

The most important preprocessing decision concerns what counts as a search state. Walking does not change the puzzle; only pushes do. Two positions with the same crates whose keepers can walk to each other are therefore the same problem. Figure 4.5 compares the two views. On the left, a step-level search treats three keeper squares as three different nodes for the same puzzle. On the right, the push-level search identifies a state by its crates plus the region the keeper can reach (shaded blue), and the actions are the eight pushes marked by green arrows. When a state is expanded, every way of arriving at it is marked as closed, so no state is expanded, or even flood-filled, twice. The Review 1 version of the system did not do this, and fixing it reduced Breadth-First Search on the Storeroom level from 62,560 expanded positions to 9,952, and A* from 1,000 to 67.

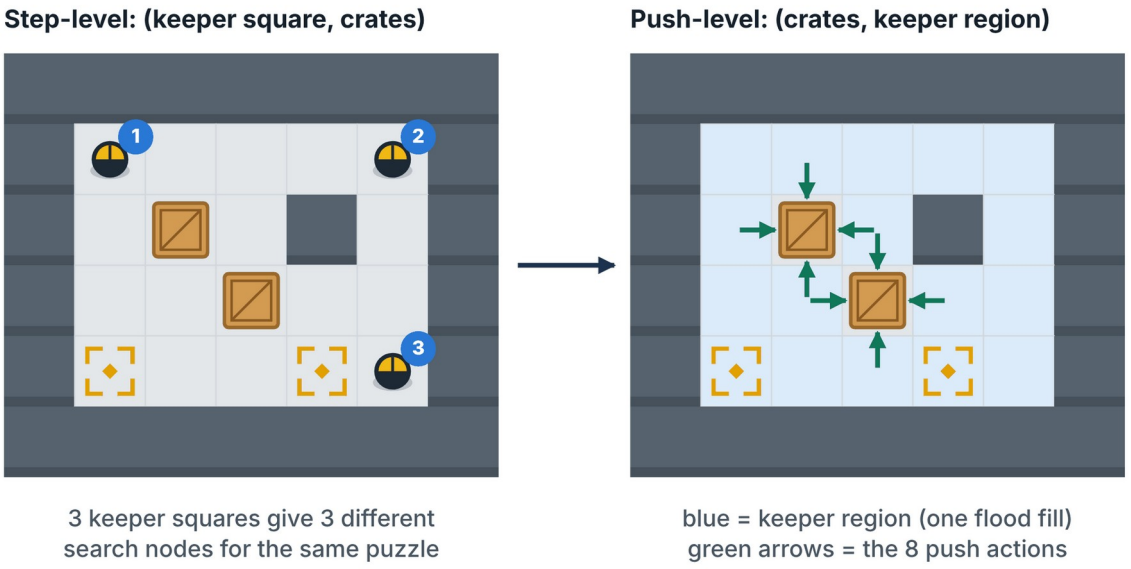


Figure 4.5 From Step-Level to Push-Level States by Keeper-Region Normalisation

4.2.1 Algorithms / Methodology

The problem is defined formally as follows. Let W, F and G be the sets of wall, floor and goal squares. A state is s = (B, R), where B ⊂ F is the set of squares holding crates and R is the region the keeper can reach. The actions in s are the pushes (b, d) with b ∈ B, d ∈ {U, D, L, R}, b − d ∈ R and b + d ∈ F \ B. A push moves the crate to b + d and leaves the keeper on b. The goal test is B = G. Each push costs 1, so the path cost g(n) is the number of pushes, and all four algorithms minimise it.

Algorithm 1: Breadth-First Search. BFS expands states level by level, in order of their number of pushes from the start. Since every push has the same cost, the first goal state generated already has the fewest pushes, so BFS can apply the goal test as soon as a state is generated. BFS is complete and optimal, but it has no sense of direction, and its time and memory grow as O(b^d) for branching factor b and solution depth d [6]. Algorithm 4.1 gives the procedure.

Algorithm 4.1 ***Breadth-first search over push-level states (solvers/bfs.py)***

```
function BFS(level, start):
```

```
    if start.boxes = goals: return []
    queue <- [start];  queued <- {start};  closed <- {}
    while queue not empty:
        s <- queue.pop_front()
        if s in closed: continue                # same crates, same region
        R <- flood_fill(s.keeper, s.boxes)
        closed <- closed + arrivals(s.boxes, R)   # every keeper cell next to a crate
        for (box, d) in legal_pushes(s.boxes, R):
            c <- push(s, box, d)
            if c in queued or c in closed: continue
            parent[c] <- (s, (box, d));  queued <- queued + {c}
            if c.boxes = goals: return trace(parent, c)     # goal test on generation
            queue.push_back(c)
        every 32 expansions: yield                # let the caller pause, draw or
race
    return UNSOLVABLE
```

Algorithm 2: A* search. A* always expands the state with the lowest value of

$$f(n) = g(n) + h(n) \tag{4.1}$$

Here g(n) is the number of pushes made so far and h(n) is an estimate of the number still needed [7]. When two states have the same f, the one with the smaller h, which appears closer to the goal, is chosen first. Unlike BFS, A* applies the goal test when a state is expanded, and it skips stale heap entries (states already expanded, or reached again later with a larger g). The first A* rung uses the Manhattan heuristic of Equation 4.2.

$$h_m(B) = \Sigma\, b \in B \;\; \min g \in G \; (\, |x_b - x_g| + |y_b - y_g| \,) \tag{4.2}$$

This heuristic is admissible. A push moves exactly one crate by exactly one square, so it can reduce that crate's Manhattan distance to any goal by at most one. A crate that is k squares from its nearest goal therefore needs at least k more pushes, and the sum over all crates can never exceed the real remaining cost. It is also consistent, because one push changes only one term of the sum, and by at most one, so $h(n) \le 1 + h(n')$ for every successor n'. With a consistent heuristic, the first time A* expands a state it has already found the cheapest path to it, so the push count it returns is optimal and no state has to be expanded twice [7]. Algorithm 4.2 gives the procedure, including the optional deadlock check.

Algorithm 4.2 ***A* with optional deadlock pruning (solvers/astar.py)***

```
function ASTAR(level, start, h, deadlocks = none):
    if h(start) undefined or deadlocks.check(start): return UNSOLVABLE
    frontier <- heap[(h(start), h(start), start)];  best_g[start] <- 0
    while frontier not empty:
        (f, hs, s) <- frontier.pop_min()          # ties -> smaller h
        if s in closed or g(s) > best_g[s]: continue
        if s.boxes = goals: return trace(parent, s)   # goal test on expansion
        R <- flood_fill(s.keeper, s.boxes);  closed <- closed + arrivals(s.boxes, R)
        for (box, d) in legal_pushes(s.boxes, R):
            c <- push(s, box, d)
            if c in closed or best_g.get(c, inf) <= g(s) + 1: continue
```

```
            if deadlocks and (rule <- deadlocks.after_push(c.boxes, box + d)):
                pruned[rule] += 1;  continue       # never enters the frontier
            hc <- h(c.boxes)
            if hc undefined: pruned["matching"] += 1;  continue
            best_g[c] <- g(s) + 1;  parent[c] <- (s, (box, d))
            frontier.push((g(s) + 1 + hc, hc, c))
        every 32 expansions: yield
    return UNSOLVABLE
```

Algorithm 3: Deadlock pruning. The deadlock layer is a small set of rules, each of the form “IF this pattern appears around a crate THEN the position cannot be solved”. Figure 4.6 shows one example of each of the four rules, with the crates in deadlock outlined in red: a crate in a corner, a crate on a wall run that has no goal (marked in orange), a crate on a square from which no goal can be reached, and a block of four crates that hold each other in place.

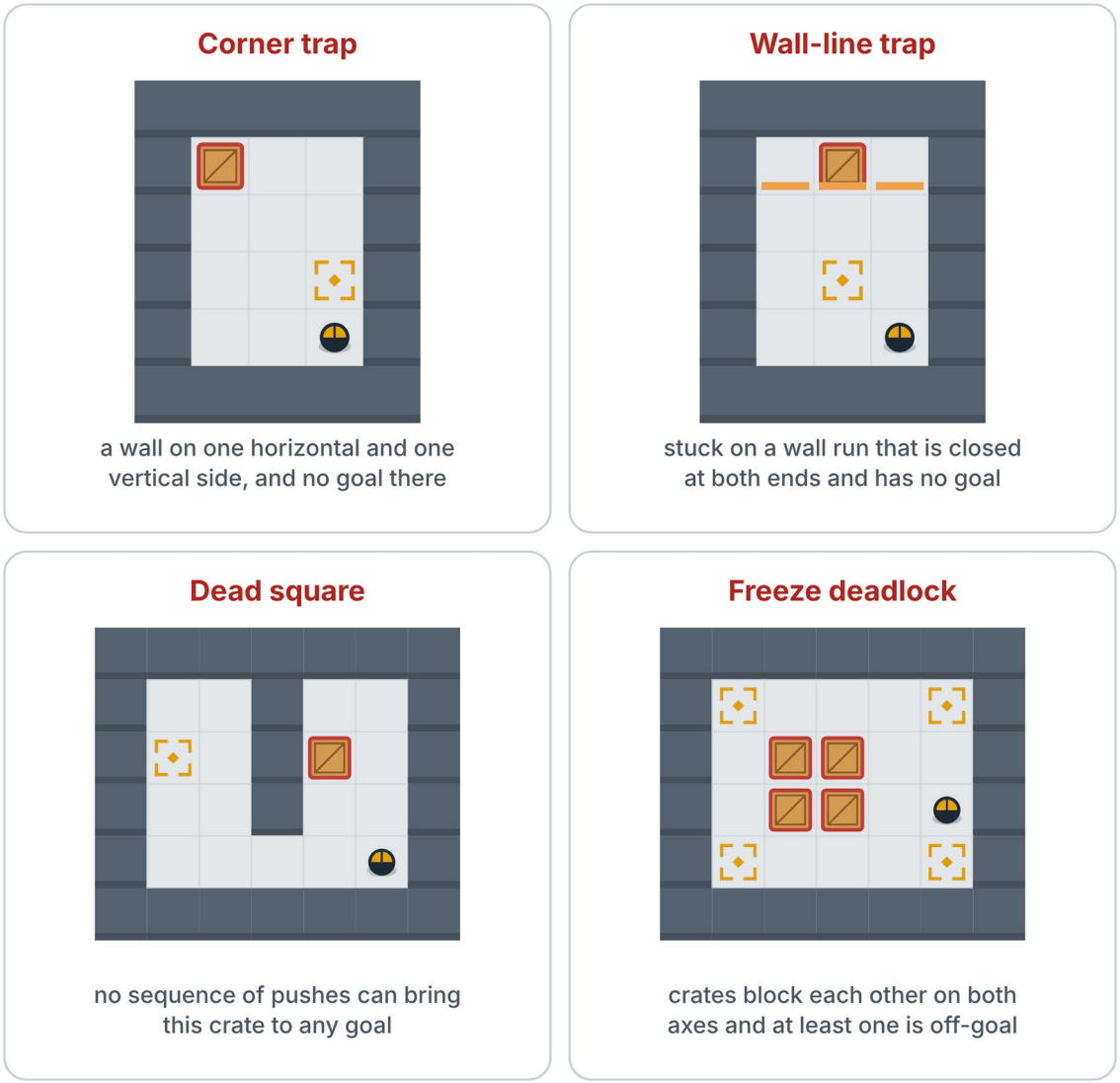


Figure 4.6 The Four Deadlock Rules (red outline = crate in deadlock)

Three of the rules are static. They depend only on the level, so they are looked up in the precomputed dead-square map in constant time. The fourth, the freeze rule, depends on the positions of the other crates and is checked only for the crate that has just moved. A crate is frozen when it is blocked along both axes. An axis is blocked by a wall on either side, by dead squares on both sides, or by a neighbouring crate that is itself frozen. During the check, a crate that is already part of the current chain of reasoning is treated as a wall, which allows a ring or block of crates that hold each other in place to be recognised. If any crate in a frozen group is off its goal, the position is a deadlock.

These rules fire only on positions that are truly lost. A crate in a corner or on a dead wall line can never leave it, a no-path square is unreachable backwards from every goal by construction, and a frozen group can never move again. Pruning therefore removes only dead ends and never a solution, which is why A* with deadlock pruning returns the same optimal push count as A*. The test suite confirms this: on hundreds of random positions, every position flagged by the rules was shown to be unsolvable by exhaustive BFS.

Algorithm 4: Matching heuristic. Manhattan distance has two weak points. It ignores walls, and it allows several crates to count the same goal. Figure 4.7 shows the second weakness on a small position. With Manhattan distance, both crates measure their distance to the same nearest goal, which gives h = 2 + 3 = 5. The matching heuristic assigns each crate a different goal and gives h = 2 + 4 = 6, which is closer to the true cost.

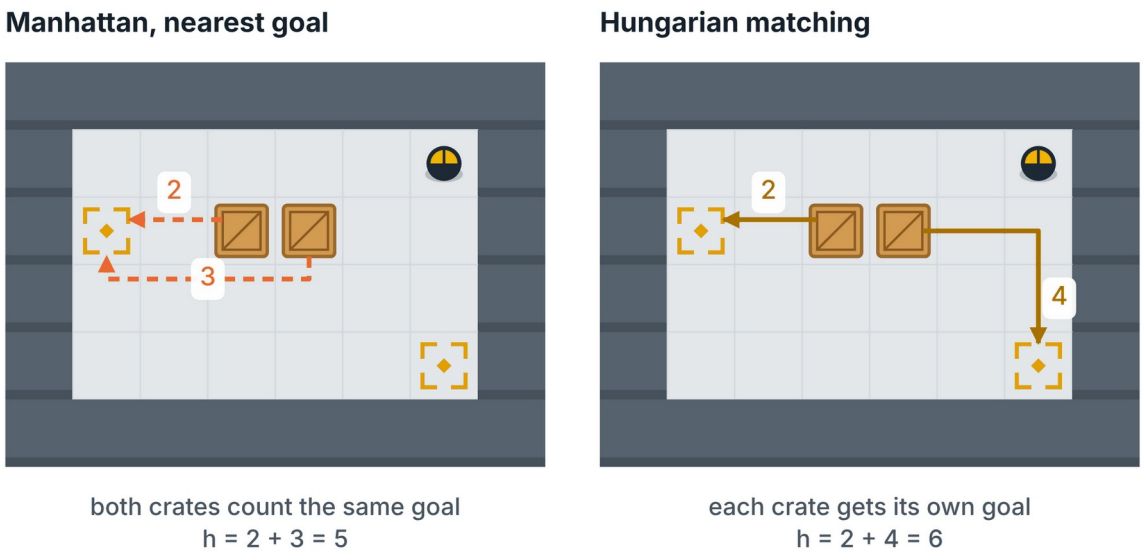


Figure 4.7 Manhattan versus Hungarian Matching Heuristic on the Same Position

The matching heuristic uses the true push distance d(b, g) from the pull analysis, and every crate must be matched to a different goal. Its value is the cost of the cheapest such matching, given by Equation 4.3.

$$h^{M}(B) = \min \text{ over bijections } \sigma: B \to G \text{ of } \Sigma\, b \in B\ d(b, \sigma(b)) \qquad (4.3)$$

It is computed with the Hungarian algorithm [24] in $O(n^3)$ time for n crates, and the result is cached for each set of crate positions. Each crate needs at least d(b, σ(b)) pushes in any solution and two crates cannot share a goal, so h^M is admissible. It is consistent for the same reason as h_m, since one push moves one crate by one square. It is also never smaller than h_m, so A* with this heuristic never expands more states. A useful side effect is that when no finite matching exists, the position cannot be solved, which acts as a fifth deadlock check at no extra cost.

To make the search pausable and the comparison fair, each solver is a generator that yields every 32 expansions. A `Search` object runs it for a slice of time, checks the node and time limits and finally builds a result record. A `Race` runs four searches in equal time slices, and the benchmark runs one race per level and adds the saving compared with BFS to every result. Table 4.4 summarises the properties of the four algorithms.

Table 4.4 Properties of the Four Algorithms

Algorithm	Complete	Optimal (pushes)	Heuristic	Pruning	Extra cost per node
BFS	Yes	Yes	None (h = 0)	None	Flood fill + push generation
A*	Yes	Yes (consistent h)	Manhattan, O(k) per state	None	+ heap operation
A* + Deadlock	Yes	Yes (sound rules)	Manhattan	Corner, wall-line, no-path, freeze	+ O(1) lookup, local freeze check
A* + DL + Matching	Yes	Yes (admissible, consistent)	Push distance + Hungarian, $O(k^3)$, cached	As above + no assignment	+ assignment on new crate sets

4.2.2 Results and Discussion

Correctness was checked first. The test suite has 59 tests, and all of them pass in about 2.3 s. Besides ordinary unit tests for parsing, moves and the bridge, it contains property-based fuzz tests. On hundreds of random positions in four test rooms, all four algorithms must give the same push count, or all must report “unsolvable”. Neither heuristic may ever exceed the true optimum, every position flagged by a deadlock rule must be unsolvable according to exhaustive BFS, and the Hungarian implementation must match brute force on random matrices of size 1 to 5. Table 4.5 lists the test files and what each one verifies.

Table 4.5 Automated Test Suite Results

Test file	Test classes	Tests	What is verified	Result
test_engine.py	ParseLevel, Moves	10	Parsing, rejection of invalid levels, pushes, blocking, LURD replay	Pass
test_solvers.py	Assignment, Solving, Heuristics, DeadlockRules, Comparison	18	Optimality agreement (fuzz), admissibility, rule soundness, Hungarian vs brute force, races	Pass
test_puzzles.py	Library, Generator	5	Every level valid and solvable; generated levels reproducible and solvable	Pass
test_analytics.py	BenchmarkRun	4	One row per level and algorithm; reports; summary	Pass
test_bridge.py	Playing, Jobs, Editor, JsonDoor	19	Timeline, coach, hints, races, benchmark streaming, validation errors, JSON door	Pass
test_site.py	Site	3	Bundle complete; published site equals sources; runtime vendored	Pass
Total		59	Ran in 2.3 s	All pass

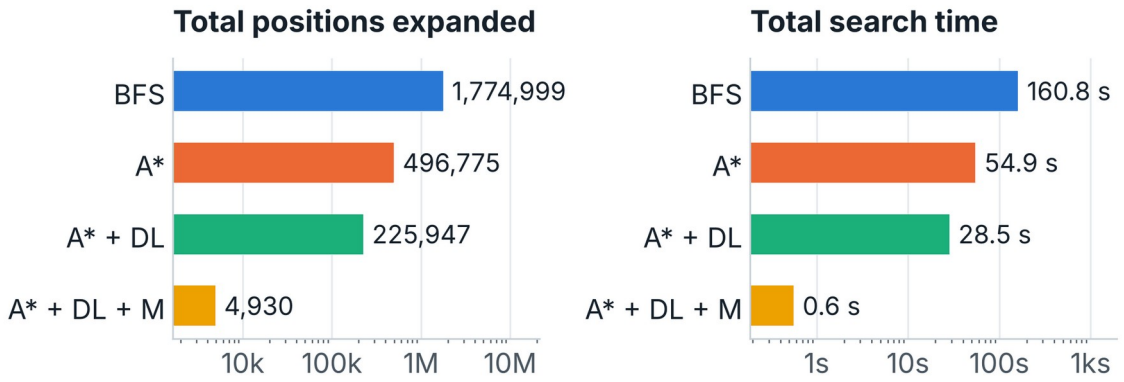
The benchmark agreed with the tests. On every one of the 34 levels, every algorithm that finished reported the same number of pushes.

Table 4.6 gives the totals over all 34 levels. BFS solved 27 levels and stopped on seven, four at the 20-second limit and three at the 200,000-position limit. Each rung then removes a large part of the remaining work. In total, A* expands 72 % fewer positions than BFS, deadlock pruning roughly halves the work of A*, and the matching heuristic cuts that by a further factor of about 46.

Table 4.6 Benchmark Summary over All 34 Levels

Algorithm	Levels solved	Positions expanded	Positions generated	Total time	Median saving vs BFS
BFS	27 / 34	1,774,999	10,888,291	160.80 s	baseline
A*	34 / 34	496,775	2,781,677	54.91 s	84.1 %
A* + Deadlock	34 / 34	225,947	1,067,097	28.48 s	92.6 %
A* + Deadlock + Matching	34 / 34	4,930	21,887	0.56 s	99.5 %

Figure 4.8 draws the same totals as bar charts on logarithmic scales, positions expanded on the left and search time on the right. The best solver needs 4,930 positions and 0.56 s for the whole set of levels, which is 360 times fewer positions and 287 times less time than BFS. Since BFS was stopped early on its hardest levels, the true factors are even larger.



All 34 levels; BFS solved 27/34 within budget, the three A* variants 34/34. Log scales.

Figure 4.8 Total Positions Expanded and Total Search Time (log scales)

Figure 4.9 shows the positions expanded on each level of the two hard packs, with one row per level and one marker per algorithm. The four markers appear in the same order on every level, BFS on the right and the matching rung on the left, often a thousand times apart. Two results stand out. On Inventory Day and Peak Season, plain A* needed 156,001

and 125,407 positions, about as many as BFS had used when its budget ran out, because the walls in these tight rooms mislead the Manhattan heuristic. The matching heuristic, which uses real push distances, solved the same two levels with 68 and 1,415 positions. On Backlog and Rush Hour, BFS ran out of budget (hollow markers), while the best solver needed only 532 and 344 positions.



W = Warehouse, S = Showdown. Hollow marker: search stopped by the budget (lower bound).

Figure 4.9 Positions Expanded per Level, Warehouse and Showdown Packs

Table 4.7 lists the positions expanded on representative levels from each pack. The full table for all 34 levels is given in Table A.1 of the Appendix.

Table 4.7 Positions Expanded on Representative Levels

Level (pack)	Crates	Pushes	BFS	A*	A* + DL	A* + DL + M
Dead-end Lane (T)	3	15	2,046	262	42	15

Level (pack)	Crates	Pushes	BFS	A*	A* + DL	A* + DL + M
Storeroom (C)	3	12	9,952	67	65	13
Partition (C)	4	14	≥ 157,664	14	14	14
Night Shift (W)	3	18	6,189	2,377	515	20
Pallet Maze (W)	4	18	52,833	11,220	4,156	518
Inventory Day (W)	5	20	≥ 133,920	156,001	67,416	68
Tight Turn (S)	4	15	38,746	3,822	1,340	31
Dead Stock (S)	4	21	77,568	23,707	2,969	79
Backlog (S)	5	17	≥ 200,000	18,343	7,907	532
Rush Hour (S)	6	21	≥ 200,000	107,759	38,453	344

Figure 4.10 divides the count of each algorithm by the count of BFS on the 23 levels where BFS finished and needed at least 30 positions; each dot is one level and the black bar is the median. The median A* run needed 11.4 % of the positions used by BFS, A* with deadlock pruning 5.6 % and the matching rung 0.7 %. At Review 1 the target was to find the optimal solution “after exploring only about 1–2 % of the states”. The final system does better, with a median of 0.7 % on these levels and 0.5 % over all 34.

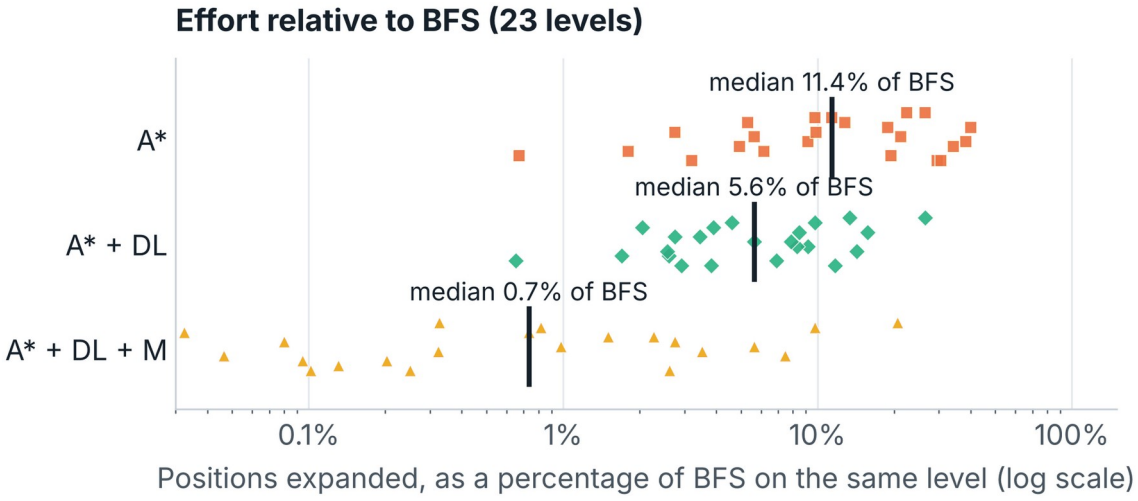


Figure 4.10 Effort Relative to BFS on Each Level (log scale)

Table 4.8 gives the totals for each pack and shows that the techniques help in different situations. In the open Classic rooms the heuristic does nearly all the work: A* needs 197 positions where BFS needed over half a million, and deadlock pruning adds little because

open rooms have few traps. In the tight Warehouse rooms the situation is reversed. Manhattan distance is weak there, and deadlock pruning almost halves the effort of A*, from 314,482 to 160,960. On both hard packs, the matching heuristic makes the largest difference.

Table 4.8 Positions Expanded and Time per Pack

Pack	BFS	A*	A* + Deadlock	A* + DL + Matching
Training (7)	2,717 · 0.07 s	313 · 0.01 s	92 · <0.01 s	48 · <0.01 s
Classic nine (9)	502,411 · 60.6 s	197 · 0.01 s	191 · 0.01 s	78 · 0.01 s
Warehouse (9)	529,916 · 51.9 s	314,482 · 34.6 s	160,960 · 23.0 s	2,092 · 0.29 s
Showdown (9)	739,955 · 48.3 s	181,783 · 20.3 s	64,704 · 5.4 s	2,712 · 0.26 s

Figure 4.11 shows how the deadlock layer works in practice. The left chart counts the pushes removed by each rule, and the right chart counts the dead squares precomputed for each static rule. Across all levels, A* with deadlock pruning removed 370,183 pushes before they reached the frontier: 230,762 by the corner rule, 85,401 by the wall-line rule, 54,012 by the freeze rule and only 8 by the no-path rule.

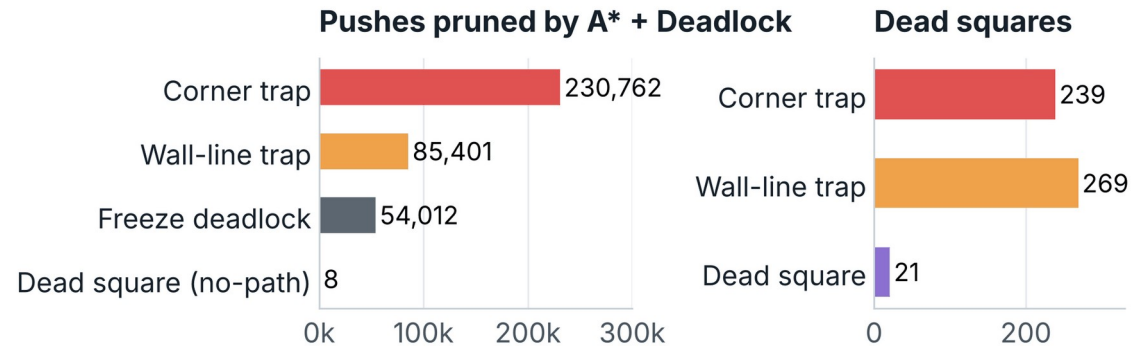
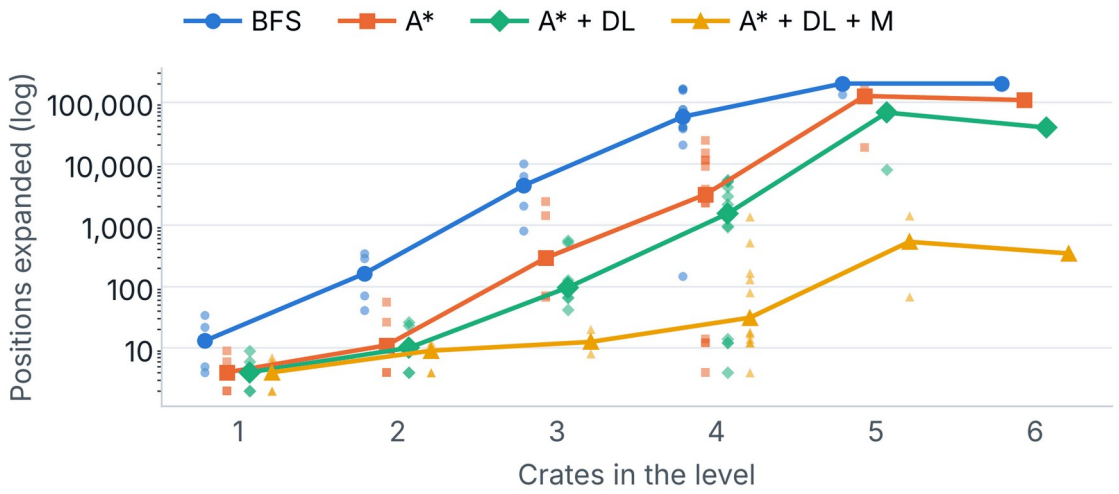


Figure 4.11 Pushes Pruned per Rule and Dead Squares per Rule over All Levels

Since the dead-square map is computed in advance, each of the three static rules costs a single set lookup. The freeze check does have a cost, which shows in the time per expanded position: about 91 µs for BFS, 110 µs for A* (because of the heap), 126 µs for A* with deadlock pruning and 114 µs for the matching rung, whose cached matchings are cheap. Peters [19] also observed that deadlock checks slow down each node. In this project the extra cost is small compared with the 55 % drop in positions expanded.

Figure 4.12 plots the search effort against the number of crates, with lines joining the medians and dots for single levels. For BFS, each extra crate costs roughly ten times more positions, and at five crates it reaches the budget. A* and A* with deadlock pruning bend the curve down, but it still rises steeply. The matching rung stays within a few hundred positions up to six crates, which is what makes instant hints possible in the browser.



Lines join the medians; dots are single levels. BFS at 5 and 6 crates is capped by the budget.

Figure 4.12 Search Effort versus Number of Crates (medians and individual levels)

All the algorithms find the same optimal number of pushes, but when there are several optimal solutions, they can take different walking routes. This happened on 21 of the 34 levels. Table 4.9 gives the move counts for the Showdown pack. A* and A* with deadlock pruning always produced exactly the same route, because pruning only removes dead ends and does not change the order in which the remaining states are expanded.

Table 4.9 Solution Length in Keeper Moves, Showdown Pack

Level	Pushes	BFS	A*	A* + DL	A* + DL + M
Aisle Nine	14	40	61	61	37
Double Stack	15	75	54	54	77
Tight Turn	15	44	45	45	46
Long Haul	19	72	73	73	77
Bottleneck	16	73	68	68	71
Four-way Stop	18	45	63	63	35
Dead Stock	21	104	117	117	104

Level	Pushes	BFS	A*	A* + DL	A* + DL + M
Backlog	17	budget	88	88	93
Rush Hour	21	budget	81	81	87

Figures 4.13 to 4.20 are screenshots of the running system. Figure 4.13 shows the Play view on the Storeroom level. The board is on the left, and the sidebar shows the pushes, moves and time of the current run, the AI's best result to beat, the overlay switches and the solver controls.

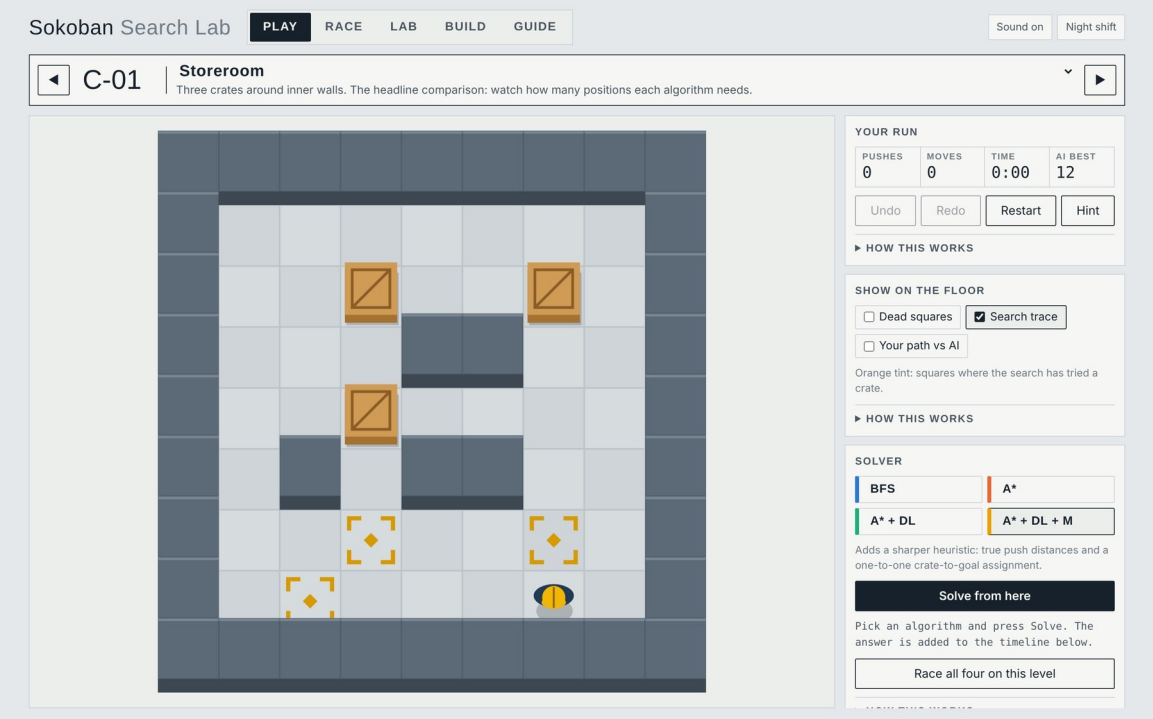


Figure 4.13 Play View of the Web Application (Storeroom, Classic Pack)

Figure 4.14 shows the dead-square overlay on Pallet Maze. Every square on which a crate must never be placed is hatched in red, and pointing at a square shows which rule makes it dead.

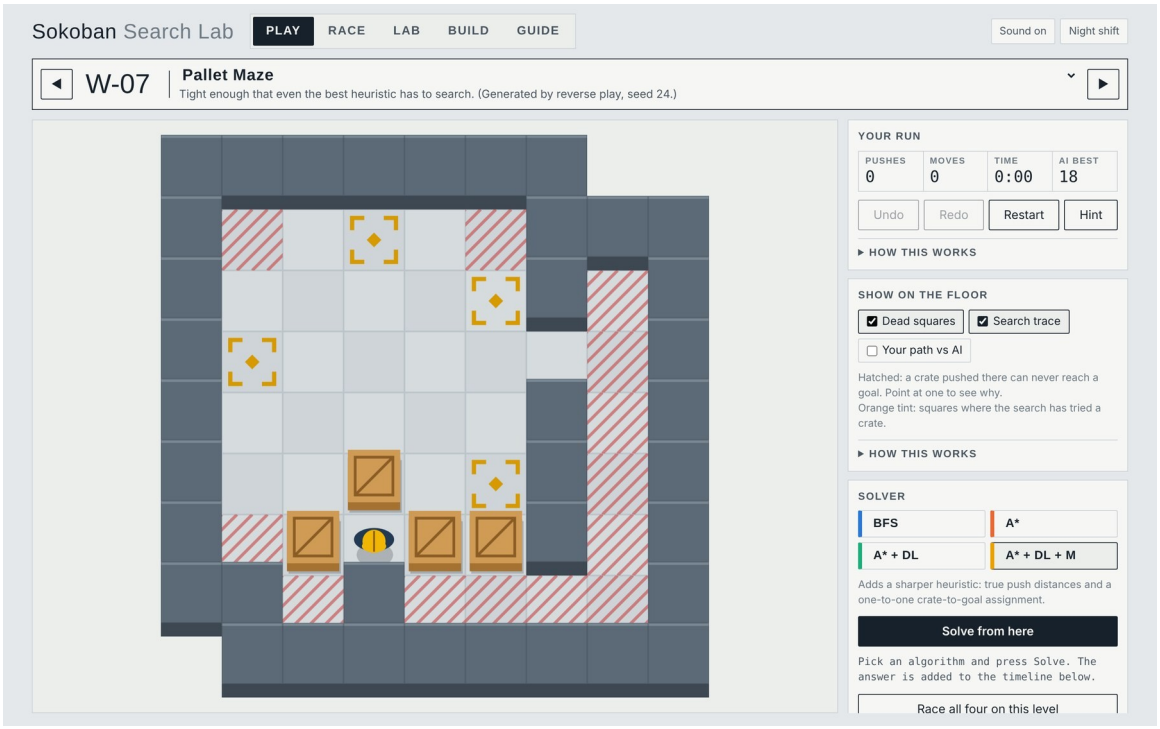


Figure 4.14 Dead-Square Overlay on Pallet Maze

Figure 4.15 shows the deadlock coach. After the player pushes a crate into a corner on the Corner Trap level, the crate is outlined in red and a banner names the rule that was broken.

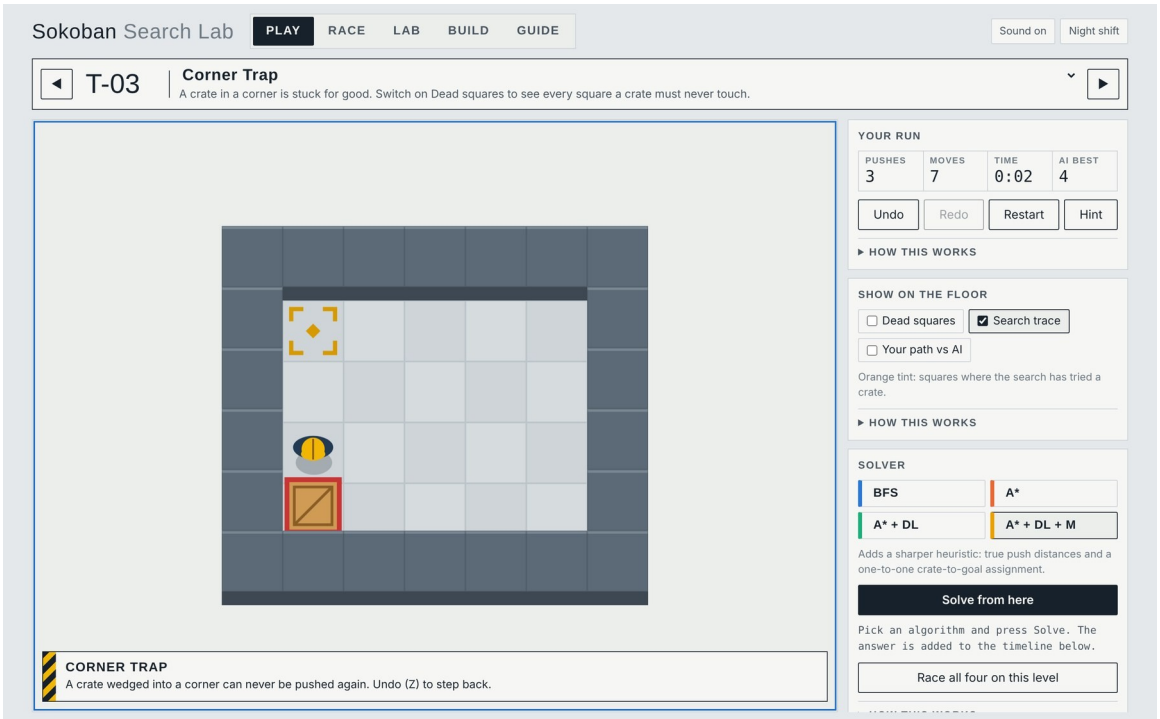


Figure 4.15 Deadlock Coach Naming a Corner Trap

Figure 4.16 shows a hint on the Storeroom level. The best solver searches from the player's current position, and the dotted path and arrow mark the walk and the next push of an optimal solution.

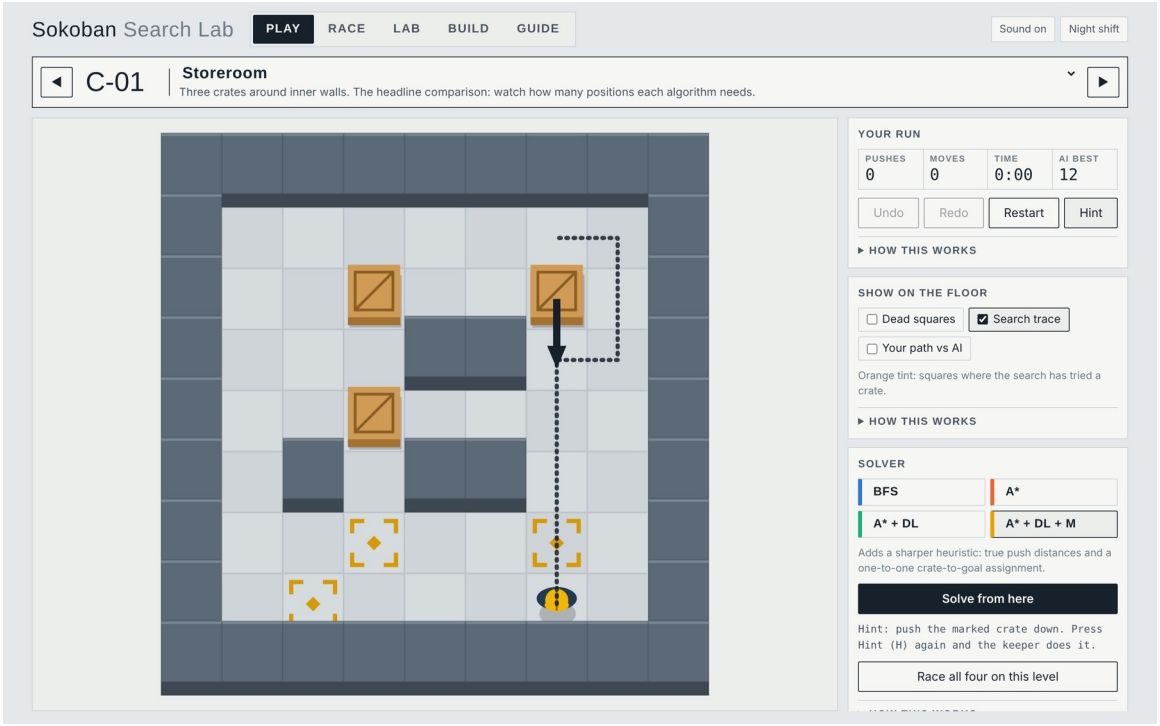


Figure 4.16 AI Hint Marking the Next Optimal Push

Figure 4.17 shows BFS while it is still searching on Tight Turn. The orange tint marks the squares where the search has tried placing crates, and a ghost shows the position currently being examined.

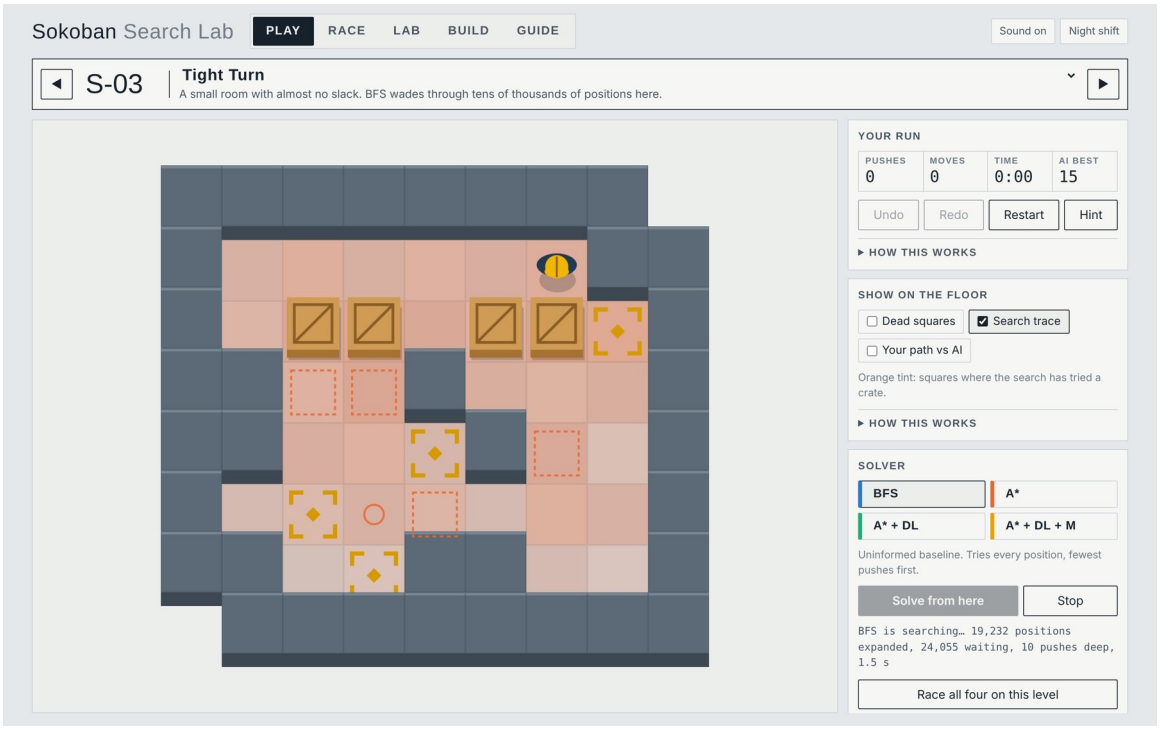


Figure 4.17 BFS Search in Progress with the Search-Trace Overlay

Figure 4.18 shows the same level after it was solved by the matching rung. The solution is added to the timeline, where it can be replayed, stepped through or scrubbed, and the readout reports the positions expanded and the pushes pruned by each rule.

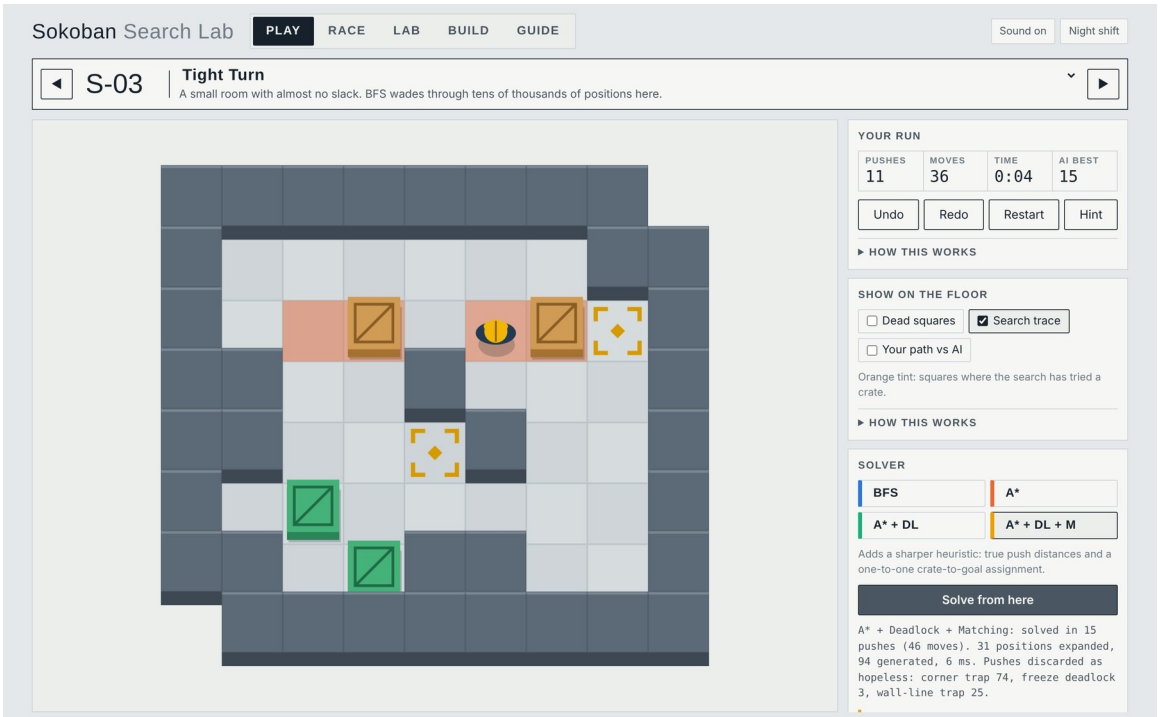


Figure 4.18 Solved Level with Timeline and Search Statistics

Figure 4.19 shows the Race view, which runs all four algorithms on the same level at the same time. On Tight Turn in the browser, BFS expanded 38,746 positions in 2.6 s, A* 3,822 in 351 ms, A* with deadlock pruning 1,340 in 110 ms and the matching rung 31 in 5 ms. That is about 1,250 times less work than BFS, and all four found the same 15-push solution.

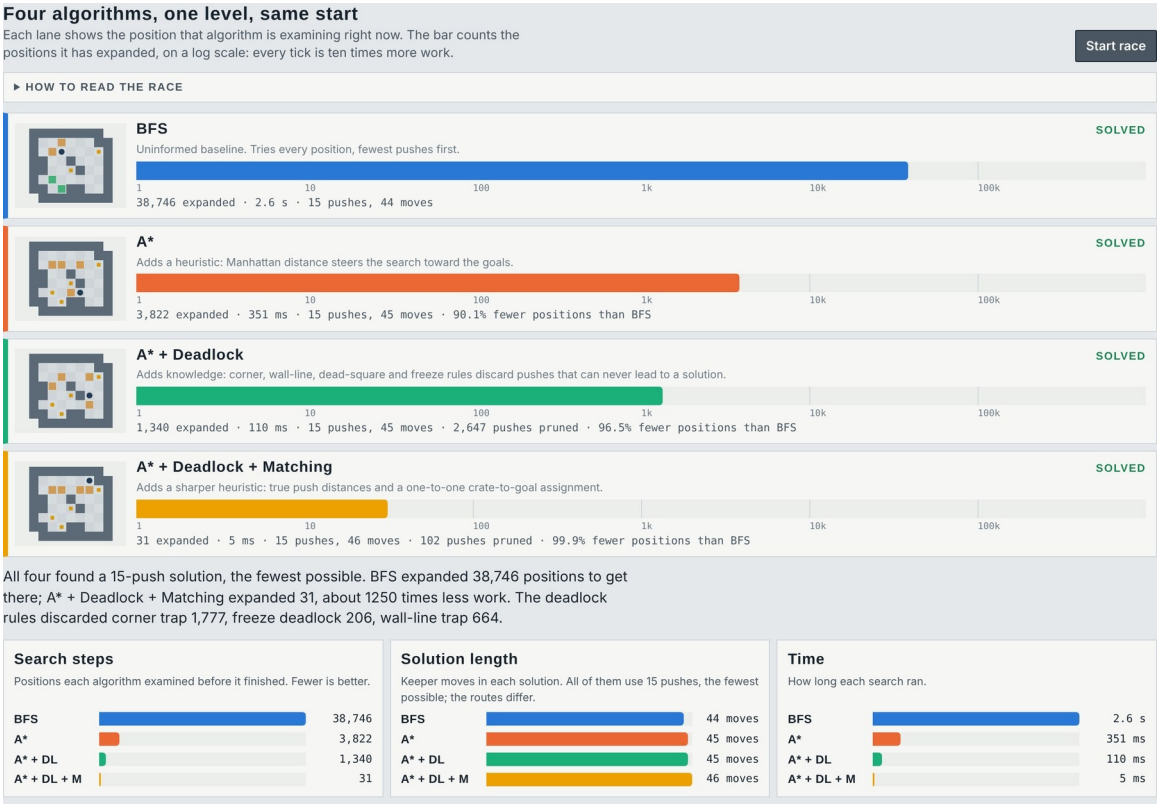


Figure 4.19 Race View: Four Algorithms on Tight Turn

Figure 4.20 shows the Lab view, which runs the benchmark inside the browser. The summary tiles give the totals for each algorithm, the matrix shows one row per level, and the results can be copied as CSV or Markdown.

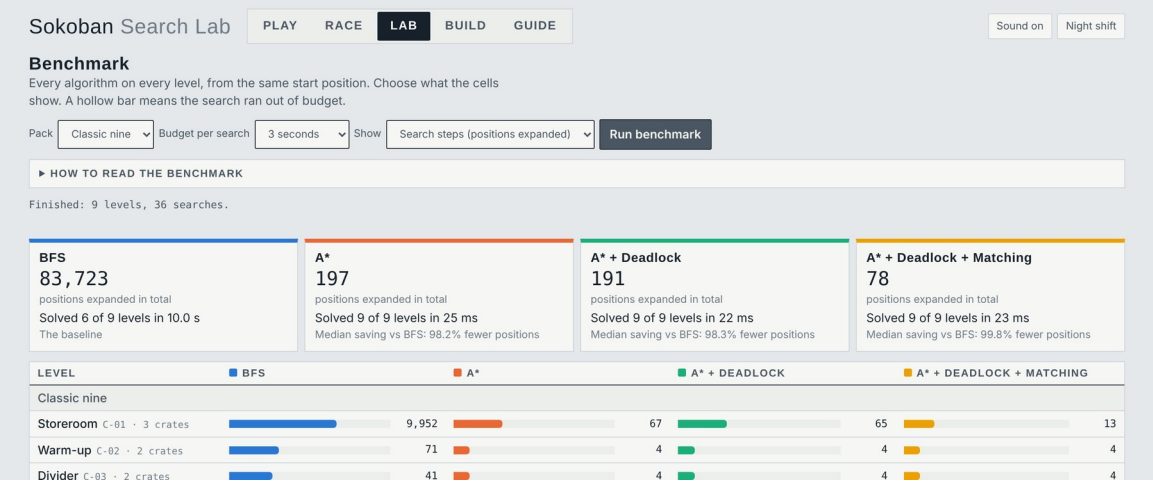


Figure 4.20 Lab View: Benchmark of the Classic Pack in the Browser

Several of these results were obtained only after the Review 1 version was reworked. A review of that version found eight problems, and each was fixed and covered by a test. The largest was the state definition described in Section 4.2. In addition, the move count had been called optimal although the search minimised pushes, and the deadlock rules had been re-derived for every crate in every state, so on eight of the nine levels of that time they pruned nothing and A* with deadlock pruning ran slower than plain A*. Some early speed-ups, such as “39,121,151x faster”, came from dividing by a timer that read zero, and a level was accepted whenever it had as many crates as goals. Finally, the first interface was a desktop pygame window, which was replaced by the web application planned at the zeroth review.

Overall, the results support the main idea of the project. Even on a PSPACE-complete puzzle, classical search becomes practical once blind exploration is replaced, step by step, by an admissible heuristic, correct domain knowledge and then a stronger heuristic. Since each rung adds only one idea and all rungs return the same optimal cost, each improvement can be attributed to the idea that caused it. Three limitations remain. First, the levels are small to medium in size, with at most six crates and 94 floor squares, and competition-size levels would need pattern databases, macro moves and goal-room analysis [4], [8]. Second, the times come from single runs on one machine, which is why positions expanded, a measure that does not depend on the machine, is used as the main measure. Third, the BFS figures on the seven levels where it stopped early are lower bounds, so the real savings are larger than those reported.

CHAPTER 5

CONCLUSION

This project designed, built and evaluated Sokoban Search Lab, a heuristic and deadlock-aware search system for solving Sokoban. The puzzle was modelled as a deterministic, push-level state space in which a state is the set of crates together with the region the keeper can reach. On this model, four optimal solvers were built as a ladder: BFS, A* with the Manhattan heuristic, A* with deadlock rules, and A* with a Hungarian matching heuristic based on true push distances. The admissibility and consistency of the heuristics were argued formally and confirmed by fuzz tests, and every deadlock rule was checked against exhaustive search.

On the 34 levels, all algorithms agree on the optimal push count. The search effort, however, drops from 1,774,999 positions and 160.8 s for BFS, which solved 27 levels, to 4,930 positions and 0.56 s for the full system, which solved all 34. The median saving over BFS is 84.1 % for A*, 92.6 % with deadlock pruning and 99.5 % with the matching heuristic, which exceeds the target set at Review 1. The deadlock rules removed 370,183 hopeless pushes and helped most in tight rooms, while in open rooms the heuristics did most of the work.

The same Python modules run from the command line, in the tests, and in a static web page through Pyodide and WebAssembly. The page offers play with a deadlock coach, optimal hints, animated solutions, live races between the algorithms, a benchmark lab and a level editor with a generator that produces only solvable levels. On the security side, the project defined security requirements, built a STRIDE threat model and implemented nine controls, among them input validation, safe output encoding, a restricted interface, limits on computation, sandboxed execution and a verified static deployment.

The project also showed the importance of checking every reported number. The timer error and the state definition corrected after Review 1 had both made the results look very different from the truth, and they were found only because every claim was tested.

FUTURE WORK (PHASE II)

- Add more deadlock knowledge (corral and bipartite deadlocks), tunnel and goal-room macros and pattern-database heuristics [13], [8], [14], and evaluate on standard sets such as XSokoban and Microban.
- Add IDA* [25], which needs much less memory, and weighted A* modes that trade some solution quality for speed, and measure that trade-off.
- Move long searches to a Web Worker so that the solver never shares a thread with the interface.
- Close the remaining security gaps: a maximum level size, a Content Security Policy meta tag with `'wasm-unsafe-eval'`, Subresource Integrity for the runtime files, and a protected release branch with signed commits.
- Carry out a small study with students to find out whether the race, the overlays and the coach improve the understanding of informed search.

REFERENCES

[1] J. Culberson, “Sokoban is PSPACE-complete,” Dept. of Computing Science, University of Alberta, Edmonton, Tech. Rep. TR97-02, 1997.

[2] D. Dor and U. Zwick, “SOKOBAN and other motion planning problems,” *Computational Geometry*, vol. 13, no. 4, pp. 215–228, 1999.

[3] A. Junghanns and J. Schaeffer, “Sokoban: Evaluating standard single-agent search techniques in the presence of deadlock,” in *Advances in Artificial Intelligence (Canadian AI 1998)*, LNCS 1418. Berlin, Germany: Springer, 1998, pp. 1–15.

[4] A. Junghanns and J. Schaeffer, “Sokoban: Enhancing general single-agent search methods using domain knowledge,” *Artificial Intelligence*, vol. 129, no. 1–2, pp. 219–251, 2001.

[5] S. Racanière *et al.*, “Imagination-augmented agents for deep reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[6] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[7] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[8] A. G. Pereira, M. Ritt, and L. S. Buriol, “Optimal Sokoban solving using pattern databases with specific domain knowledge,” *Artificial Intelligence*, vol. 227, pp. 52–70, 2015.

[9] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: https://owasp.org/Top10/

[10] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: Wiley, 2014.

[11] United Nations General Assembly, “Transforming our world: The 2030 Agenda for Sustainable Development,” Resolution A/RES/70/1, 2015.

[12] A. Botea, M. Müller, and J. Schaeffer, “Using abstraction for planning in Sokoban,” in *Computers and Games (CG 2002)*, LNCS 2883. Berlin, Germany: Springer, 2003, pp. 360–375.

[13] J. C. Culberson and J. Schaeffer, “Pattern databases,” *Computational Intelligence*, vol. 14, no. 3, pp. 318–334, 1998.

[14] A. G. Pereira, R. Holte, J. Schaeffer, L. S. Buriol, and M. Ritt, “Improved heuristic and tie-breaking for optimally solving Sokoban,” in *Proc. 25th Int. Joint Conf. Artificial Intelligence (IJCAI)*, New York, NY, USA, 2016, pp. 662–668.

[15] L. Orseau, L. H. S. Lelis, T. Lattimore, and T. Weber, “Single-agent policy tree search with guarantees,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[16] A. Venkatesan, A. Jain, and R. Grewal, “AI in game playing: Sokoban solver,” arXiv:1807.00049, 2018.

[17] Y. Shoham and J. Schaeffer, “The FESS algorithm: A feature based approach to single-agent search,” in *Proc. IEEE Conf. Games (CoG)*, 2020.

[18] Y. Shoham and G. Elidan, “Solving Sokoban with forward-backward reinforcement learning,” in *Proc. 14th Int. Symp. Combinatorial Search (SoCS)*, 2021. arXiv:2105.01904.

[19] N. S. Peters, “Solving Sokoban efficiently: Search tree pruning techniques and other enhancements,” B.Sc. thesis, School of Electrical Engineering and Computer Science, Technische Universität Berlin, Germany, 2023.

[20] L. Lehnert *et al.*, “Beyond A*: Better planning with transformers via search dynamics bootstrapping,” arXiv:2402.14083, 2024.

[21] Pyodide contributors, “Pyodide: A Python distribution for the browser based on WebAssembly,” version 314.0.7, 2026. [Online]. Available: https://pyodide.org

[22] D. S. Bento, A. G. Pereira, and L. H. S. Lelis, “Procedural generation of initial states of Sokoban,” in *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Macao, China, 2019.

[23] W3C, “Content Security Policy Level 3,” W3C Working Draft. [Online]. Available: https://www.w3.org/TR/CSP3/

[24] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[25] R. E. Korf, “Depth-first iterative-deepening: An optimal admissible tree search,” *Artificial Intelligence*, vol. 27, no. 1, pp. 97–109, 1985.

[26] Mettu Mani Chandhan Sai and Mebish V D, “Sokoban Search Lab,” source code repository, 2026. [Online]. Available: https://github.com/MettuManiChandhanSai/sokoban-search-lab

APPENDIX

A1 RUNNING THE SYSTEM

The project needs only Python 3.9 or newer, and nothing has to be installed. The hosted version runs at https://mettumanichandhansai.github.io/sokoban-search-lab/ and the source code is available on GitHub [26]. The commands below build and serve the site, run the benchmark and run the tests.

```
git clone https://github.com/MettuManiChandhanSai/sokoban-search-lab.git
cd sokoban-search-lab
python run_game.py          # build the site and play at http://localhost:8000
python run_benchmark.py     # every algorithm on every level: table, CSV, Markdown
python run_benchmark.py --pack showdown --node-limit 200000 --time-limit 20
python -m unittest          # 59 tests
```

A2 LEVEL FILE FORMAT

A level is a text file `puzzles/<pack>/<nn_name>.txt`. The first comment line is the title and the next one is a note shown to the player. The Storeroom level is shown below as an example.

```
; Storeroom
; Three crates around inner walls. The headline comparison.
#########
#       #
#  $  $ #
#   ##  #
#  $    #
# # ##  #
#  .  . #
# .   @ #
#########
```

A3 SAMPLE CODE: DEADLOCK RULES AND THE FREEZE TEST

The listing below shows how the dead-square map is built from the pull analysis and how each push is checked against the static rules and the freeze rule.

```
class Deadlocks:
    def __init__(self, level):
        alive = set().union(*push_distances(level).values())   # pull analysis
        corners, lines = corner_squares(level), wall_line_squares(level)
        self.dead = {cell: "corner" if cell in corners
                     else "wall-line" if cell in lines else "no-path"
                     for cell in level.floor - alive}

    def after_push(self, boxes, box):
        if box in self.dead:                       # static rules: O(1)
            return self.dead[box]
        cluster = self.frozen(boxes, box)          # dynamic rule
```

```
        if cluster and not cluster <= self.level.goals:
            return "freeze"
        return None
```

A4 SAMPLE CODE: HUNGARIAN ASSIGNMENT FOR THE MATCHING HEURISTIC

The listing below shows how the matching heuristic combines the push distances with the Hungarian algorithm and caches the result for each set of crate positions.

```
def matching(level):
    dist = push_distances(level)                  # {goal: {cell: pushes}}
    rows = {cell: [dist[g].get(cell, UNREACHABLE) for g in level.goals]
            for cell in level.floor}
    cache = {}
    def h(boxes):
        if boxes not in cache:
            cost = min_assignment([rows[b] for b in boxes])   # O(n^3)
            cache[boxes] = cost if cost < UNREACHABLE else None
        return cache[boxes]                       # None = no assignment = deadlock
    return h
```

A5 COMPLETE BENCHMARK RESULTS (POSITIONS EXPANDED)

Table A.1 lists the positions expanded by every algorithm on all 34 levels, with a limit of 200,000 positions and 20 s per search. The sign “≥” marks a search that was stopped by the limit, and the push column is the optimal number of pushes found by every algorithm that finished.

Table A.1 Positions Expanded on All 34 Levels

Level	Crates	Pushes	BFS	A*	A* + DL	A* + DL + M
First Push	1	2	5	2	2	2
Get Behind It	1	2	4	2	2	2
Corner Trap	1	4	13	4	4	4
Along the Wall	1	6	22	6	6	6
Side by Side	2	9	343	11	10	9
Through the Door	2	10	284	26	26	10
Dead-end Lane	3	15	2,046	262	42	15
Storeroom	3	12	9,952	67	65	13
Warm-up	2	4	71	4	4	4

Level	Crates	Pushes	BFS	A*	A* + DL	A* + DL + M
Divider	2	4	41	4	4	4
Side Rooms	3	8	3,944	71	67	8
Crossbar	4	4	145	4	4	4
Four Corners	4	12	≥ 166,656	12	12	12
Partition	4	14	≥ 157,664	14	14	14
Partition II	4	12	≥ 163,904	12	12	12
Round the Pillar	1	7	34	9	9	7
Loading Bay	2	11	161	55	23	12
Cold Store	3	12	799	320	126	12
Mezzanine	3	12	4,799	1,417	562	12
Night Shift	3	18	6,189	2,377	515	20
Cross-dock	4	18	55,497	2,954	2,163	18
Overstock	4	17	75,718	14,731	5,221	17
Pallet Maze	4	18	52,833	11,220	4,156	518
Inventory Day	5	20	≥ 133,920	156,001	67,416	68
Peak Season	5	21	≥ 200,000	125,407	80,778	1,415
Aisle Nine	4	14	20,170	2,298	928	165
Double Stack	4	15	36,971	2,273	964	35
Tight Turn	4	15	38,746	3,822	1,340	31
Long Haul	4	19	39,882	8,952	5,350	130
Bottleneck	4	16	66,692	3,293	1,715	31
Four-way Stop	4	18	59,926	11,336	5,078	1,365
Dead Stock	4	21	77,568	23,707	2,969	79
Backlog	5	17	≥ 200,000	18,343	7,907	532
Rush Hour	6	21	≥ 200,000	107,759	38,453	344

A6 ADDITIONAL SCREENSHOTS

Figure A.1 shows the Build view, where a level can be painted on a grid, validated as it is drawn and played at once, or produced by the generator from a size and a seed.

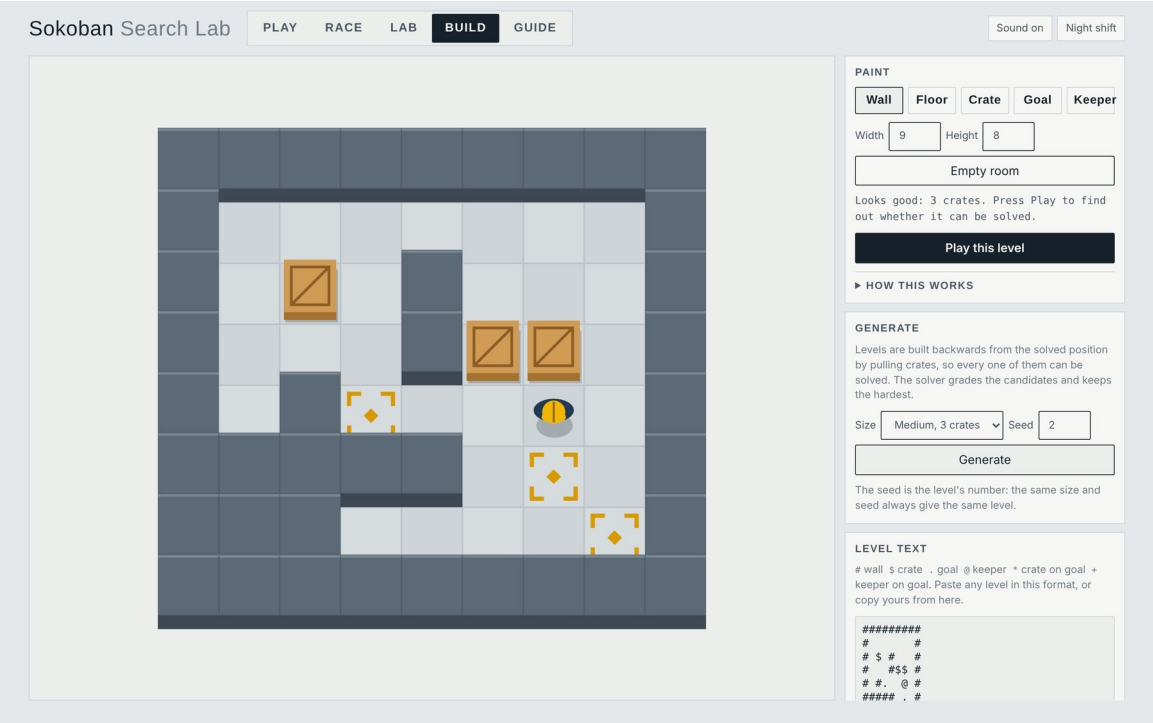


Figure A.1 Level Editor and Generator (Build View)

Figure A.2 shows the built-in guide, which explains the game, the five sections of the application, the algorithms and the meaning of each number shown on screen.

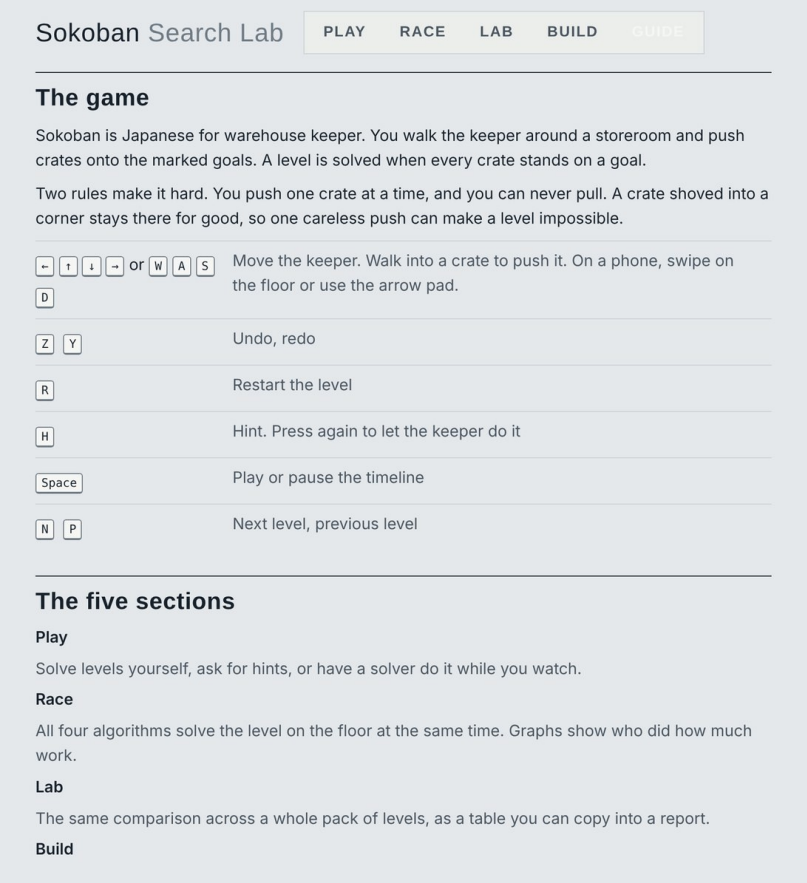


Figure A.2 Built-in Guide Explaining the Game, Algorithms and Metrics